

Cryptographic Implementations: Mathematical Theory

Cian Lalor

This document draws on J. Scholz (Rareskills), D. Dummit and R. Foote (*Abstract Algebra*), J. Silverman (*The Arithmetic of Elliptic Curves*), S. Lang (*Algebra*), C. Costello (*Pairings for beginners*), and Wikipedia¹.

Contents

1	General Theory	2
1.1	Elliptic Curves and Group Structure	2
1.2	Affine vs Projective space	3
1.3	Homomorphisms and Isomorphisms	4
1.4	Discrete Logarithm Problem	5
2	Rational Sums	7
3	ECP Matrix	8
4	ECDSA	9
4.1	Overview	9
4.2	ECDSA signature generation on secp256k1	9
4.3	ECDSA FAQs	10
5	Bilinear Pairings	10
5.1	Overview	10
5.2	Basic Theory & Definitions	10
5.3	Divisors and Rational Functions for Pairings	12
5.4	Weil and Tate Pairings	16
6	Zero-Knowledge Proof Foundations	19
6.1	Arithmetic Circuits	19
6.2	Rank-1 Constraint Systems	20
6.3	Lagrange Interpolation	21
6.4	The Schwartz-Zippel Lemma	24
6.5	Vector Equality Testing via Polynomials	24
6.6	Quadratic Arithmetic Programs	24
6.7	Building Zero-Knowledge Proofs from R1CS	26
7	Trusted Setup and QAP Evaluation	26
7.1	Trusted Setup	27
7.2	Toxic Waste and Forgery	28
7.3	Evaluating a Trusted Setup on a QAP	29

¹in addition to several articles and documents which are mentioned where relevant.

8	Groth16	30
8.1	Introducing α and β	30
8.2	Expanded QAP, Trusted Setup, and Prover	31
8.3	Public Inputs, Zero-Knowledge, and Verification	32
8.4	Separating Public and Private Inputs: γ and δ	32
8.5	Zero-Knowledge: r and s	33
8.6	Several Known Security Issues	35
8.6.1	Why proofs are accessible and why hashing does not suffice	35
8.6.2	Geometric interpretation and EVM verification	36
8.6.3	Mitigations	36
8.7	Groth16 Algorithm Summary	36
A	Proof of Fermat's Little Theorem	37
B	Rational functions on curves: vocabulary and orders	39
B.1	Rational functions on a curve	39
B.2	Zeros of multiplicity m , poles of order m , and the order $\text{ord}_P(f)$	40
B.3	Principal divisors and degree	42

1 General Theory

1.1 Elliptic Curves and Group Structure

Definition 1.1. An elliptic curve (e.g., bn128) over a field $\mathbb{F}_p$ (finite for cryptography-based applications) where p is prime can be described by a simplified equation of the Weierstrass form:

$$y^2 = x^3 + ax + b \bmod p \quad (1.1)$$

We let this equation define the curve E , and we define the set

$$E(\mathbb{F}_p) = \{(x, y) \in \mathbb{F}_p^2 : y^2 = x^3 + ax + b \bmod p\} \cup \{\mathcal{O}\} \quad (1.2)$$

to be the points, ordered pairs $(x, y) \in \mathbb{F}_p \times \mathbb{F}_p$, satisfying the equation over the prime field, where $\mathcal{O}$ is the point at infinity. We require $p \neq 2, 3$ (so the curve can be written in this short form) and $4a^3 + 27b^2 \neq 0$ (nonsingularity; see Definition 1.4).

Definition 1.2. For points $P, Q \in E(\mathbb{F}_p)$, draw the line through P and Q , or if $P = Q$ take the tangent line at P . If this line is not vertical, it meets the cubic in a third point R' , counted with multiplicity, and one defines

$$P + Q + R' = \mathcal{O}, \quad (1.3)$$

equivalently,

$$P + Q = -R'. \quad (1.4)$$

If the line is vertical, then $Q = -P$ and $P + Q = \mathcal{O}$. In the tangent case, a vertical tangent means $P = Q$ and $P = -P$ ($\implies P = (P_x, 0)$), so again $P + P = \mathcal{O}$. By definition, $\mathcal{O}$ is the identity: $P + \mathcal{O} = \mathcal{O} + P = P$.

Definition 1.3. We let a point $G = (G_x, G_y) \in E(\mathbb{F}_p)$ be our generator point for the cyclic subgroup $\langle G \rangle$. We say that subgroup $\langle G \rangle$ is generated by G and that for the least positive integer n with $nG = \mathcal{O}$ (the order of G):

$$\langle G \rangle = \{kG \mid k \in \mathbb{Z}_n\} \subseteq E(\mathbb{F}_p) \quad (1.5)$$

Observe that $15G = 2 * 2 * 2 * G + 2 * 2 * G + 2 * G + G = (2^3 + 2^2 + 2^1 + 2^0)G$, thus can be derived by doubling multiple times. Several algorithms use this concept to compute points on the curve.

We choose elliptic curves as a basis for a cryptographic primitive for several reasons including:

1. for a line intersecting two points on the curve, it can only intersect one additional point (counted with multiplicity, in projective space) as curve is cubic.
2. points will be well defined as these curves are smooth.
3. the group laws are well defined.

1.2 Affine vs Projective space

Affine space $\mathbb{F}_p^2$ is the coordinate space where points are represented by pairs (x, y) with $x, y \in \mathbb{F}_p$. Projective space extends this by adding points at infinity, represented as equivalence classes $(X : Y : Z)$ with $(X, Y, Z) \neq (0, 0, 0)$, where $(X : Y : Z) \sim (\lambda X : \lambda Y : \lambda Z)$ for $\lambda \in \mathbb{F}_p^\times$. This equivalence relation on coordinate triples means that projective points are the equivalence classes under this relation, i.e., given points in projective space we are concerned about the direction and not coordinates, so scalar multiples of triples represent the same point. And finally, to verify smoothness at the point at infinity we work in projective space.

Definition 1.4. An elliptic curve E over field $\mathbb{F}_p$ is *smooth* (non-singular) if it has no singular points. For the short Weierstrass equation in (1.1), this means that the partial derivatives of

$$F(x, y) = y^2 - (x^3 + ax + b)$$

do not vanish simultaneously at any point on the curve; equivalently, the discriminant is nonzero², meaning the Weierstrass cubic does not degenerate into a singular curve (for the short Weierstrass form, this is equivalent to $4a^3 + 27b^2 \neq 0$).

The projective closure of the curve in equation (1.1) is given by the homogenous equations:

$$Y^2Z = X^3 + aXZ^2 + bZ^3 \tag{1.6}$$

or

$$F(X, Y, Z) = Y^2Z - (X^3 + aXZ^2 + bZ^3) = 0 \tag{1.7}$$

The curve is smooth if the partial derivatives $\frac{\partial F}{\partial X}$, $\frac{\partial F}{\partial Y}$, and $\frac{\partial F}{\partial Z}$ do not all vanish simultaneously at any point $(X : Y : Z)$ on the curve where $(X : Y : Z) = (\lambda X : \lambda Y : \lambda Z)$ for any $\lambda \neq 0$ — note that each term in equation (1.7) is to the third degree.

The point at infinity is $\mathcal{O} = (0 : 1 : 0)$, which corresponds to the "direction" where the curve goes to infinity. To see why $\mathcal{O} = (0 : 1 : 0)$ and not $(0 : 0 : 0)$ or $(0 : 0 : 1)$, note that:

- $(0 : 0 : 0)$ is not a valid projective point (all coordinates cannot be zero)
- $(0 : 0 : 1)$ corresponds to the affine point $(0, 0)$, which is not at infinity
- Setting $Z = 0$ in equation (1.6) gives $0 = X^3$, so $X = 0$. Since we need $Y \neq 0$ for a valid point, we normalize to $(0 : 1 : 0)$, which represents the "vertical direction"

²See Silverman, *The Arithmetic of Elliptic Curves*, Ch. III, §1 (Weierstrass equations, invariants, and discriminant). For the short Weierstrass equation $y^2 = x^3 + Ax + B$, one has $\Delta = -16(4A^3 + 27B^2)$, and nonsingularity is equivalent to $\Delta \neq 0$.

At $(0 : 1 : 0)$, we have $\frac{\partial F}{\partial Z} = Y^2 = 1 \neq 0$, so the curve is smooth at infinity³. In projective space parallel lines meet at a point at infinity (a line exists at infinity in projective space whereas affine space has none), which is precisely what the group law requires (vertical lines intersect at $\mathcal{O}$, points require additive inverses).

The point $\mathcal{O}$ serves as the identity element for the group operation and is necessary to handle cases like vertical lines in the geometric addition law.

Remark 1.1 (Computational Benefit). Working in projective space for many presentations of an elliptic curve reduces computational cost, further motivating their use over affine space.

Examining point 3 list 1.1 more closely, we see that the binary operation of elliptic curve point addition as outlined above $\oplus : E(\mathbb{F}_p) \times E(\mathbb{F}_p) \rightarrow E(\mathbb{F}_p)$ for all points on the curve (e.g. take arbitrary $P, Q, R \in E(\mathbb{F}_p)$) satisfies the following properties:

1. Closure: $P + Q \in E(\mathbb{F}_p)$ for all $P, Q \in E(\mathbb{F}_p)$
2. Identity element exists: $P + \mathcal{O} = \mathcal{O} + P = P$
3. Additive inverse exists: $P + (-P) = \mathcal{O}$
4. Associativity: $(P + Q) + R = P + (Q + R)$

This last point can be verified using addition formulae for various points on curve, or generally by the Riemann-Roch Theorem.

1.3 Homomorphisms and Isomorphisms

Definition 1.5. For two arbitrary groups G and H , with independent operators, e.g., $(G, \otimes)$ and $(H, \oplus)$, a mapping φ such that:

$$\begin{aligned} \varphi : G &\rightarrow H \\ \varphi(x \otimes y) &= \varphi(x) \oplus \varphi(y), \forall x, y \in G \end{aligned} \tag{1.8}$$

is called a *homomorphism*. If mapping φ is also a bijection, then we say that φ is an *isomorphism*, and denote G and H as being isomorphic by $G \cong H$.

Remark 1.2 (Kernel). For a homomorphism $\varphi : M \rightarrow N$, the *kernel* of φ is

$$\ker(\varphi) = \{m \in M : \varphi(m) = e_N\}.$$

Remark 1.3. Formally, a function f from set A to set B is relation $f \subseteq A \times B$, which ensures that $\forall a \in A$, relation f contains exactly one ordered pair of the form (a, b) . When we say for function $f : A \rightarrow B$ where $f(a) = b$ we mean ordered pair $(a, b) \in f$ where f is relation. And if f is bijective, then in addition to the aforementioned, $\forall b \in B \exists$ exactly one $a \in A$ such that $(a, b) \in f$.

Theorem 1.4. The mapping φ between the cyclic group $(\mathbb{Z}_n, +)$ and the cyclic subgroup $(\langle G \rangle, +)$ of elliptic curve points, defined by:

$$\begin{aligned} \varphi : \mathbb{Z}_n &\rightarrow \langle G \rangle \subseteq E(\mathbb{F}_p) \\ k &\mapsto kG \end{aligned} \tag{1.9}$$

is an isomorphism. That is, $\mathbb{Z}_n \cong \langle G \rangle$.

³See Costello Figure 2.10 for illustrations of relationships between affine and projective spaces.

Proof. First, we show that φ is a homomorphism. Consider

$$\varphi(k_1 + k_2) = (k_1 + k_2)G = k_1G + k_2G = \varphi(k_1) + \varphi(k_2) \quad (1.10)$$

where the final equality follows from the fact that scalar multiplication is defined as repeated addition and addition of points in $E(\mathbb{F}_p)$ is associative, allowing us to regroup terms. Here $k_1 + k_2$ is taken modulo n ; this is consistent since $nG = \mathcal{O}$ gives $(k + n)G = kG$. The map is surjective because every element of $\langle G \rangle$ has the form kG by definition of the subgroup generated by G . It is injective because if $\varphi(k_1) = \varphi(k_2)$, then $(k_1 - k_2)G = \mathcal{O}$. Since G has order n , this implies $k_1 \equiv k_2 \pmod{n}$, so $k_1 = k_2$ in $\mathbb{Z}_n$. Hence φ is bijective, and therefore an isomorphism. $\square$

Corollary 1.5. Having established that this subgroup of elliptic curve points, derived by:

1. selecting an elliptic curve equation E and some prime order p
2. choosing a group generator $G \in E(\mathbb{F}_p)$
3. then a modulo n known as the order of G where n divides the curve order $|E(\mathbb{F}_p)|$, $nG = \mathcal{O}$ and often $n \approx |E(\mathbb{F}_p)|$

is isomorphic to integers modulo n , enables us to reason about elliptic curve point addition in fashion analogous to standard integer arithmetic.

1.4 Discrete Logarithm Problem

The following logarithmic relationship is well known to exist for $a, x, y \in \mathbb{R}$:

$$a^x = y \iff \log_a y = x \quad (1.11)$$

Over the real numbers, the inverse operation is given by the logarithm. In elliptic-curve cryptography one can still apply the group law repeatedly, but there is no analogous analytic inverse that recovers the scalar from a point. For a group $(G, *)$, with $a, y \in G$, one may therefore still define the forward operation by repeated composition:

$$a^x := \underbrace{a * a * \dots * a}_{x \text{ times}} = y \quad (1.12)$$

E.g. if operator $*$ is ECP point addition $+$, then renaming a, y more suggestively as $P, Q \in E(\mathbb{F}_p)$ gives

$$\begin{aligned} P^x &:= P * P * \dots * P = Q \\ P^x &:= \underbrace{P + P + \dots + P}_{x \text{ times}} = Q \\ P^x &:= xP = Q \end{aligned} \quad (1.13)$$

where xP above is typically denoted as $[x]P$.

The discrete log problem (DLP) is fundamentally harder to compute for discrete groups than real numbers as we cannot use continuous methods like calculus to determine x . In addition, Elliptic curves use scalar multiplications (repeated point addition), which is different from additive groups mod p prime. The distinction is that for an additive group $\mathbb{Z}_n$, you can say that:

$$x \cdot a = y \pmod{n} \iff x = ya^{-1} \pmod{n} \quad (1.14)$$

where as no such direct division operation exists for elliptic curves.

In classical Diffie-Hellman, the group is typically $\mathbb{F}_{p_1}^\times$ (multiplication mod p), whereas in elliptic-curve systems such as ECDSA the group is $E(\mathbb{F}_{p_2})$ (point addition); with best-known algorithms today being subexponential for finite-field DLP (index-calculus/Number Field Sieve) but exponential for generic ECDLP (Elliptic Curve Discrete Logarithm Problem) (Pollard rho), we infer that $E(\mathbb{F}_{p_2})$ can match the security of $\mathbb{F}_{p_1}^\times$ even when $p_1 \gg p_2$.

The *curve order*, $|E(\mathbb{F}_p)|$, i.e., the total number of points on the curve over $\mathbb{F}_p$ including $\mathcal{O}$, is in practice written as

$$|E(\mathbb{F}_p)| = h r, \quad (1.15)$$

where r is a large prime and h (the cofactor) is small. Given a point $P \in E(\mathbb{F}_p)$, cofactor clearing computes $Q = [h]P$, so Q lies in the prime-order subgroup (hence $[r]Q = \mathcal{O}$). The security level is therefore chosen from the DLP hardness in this subgroup of order r . More generally, when $|E(\mathbb{F}_p)|$ presented in its unique prime factorization, the ECDLP on the full group can be decomposed into DLPs on subgroups of the main group (Pohlig–Hellman) and recombined with modular arithmetic via the Chinese Remainder Theorem (CRT); the attack runtime for solving ECDLP is dominated by the largest prime-order component.

Definition 1.6. The *discrete log* of Q with respect to G exists if there is a scalar q for which $Q = qG$ (and, using standard terminology, that the *discrete log relationship* between points P and Q exists if there exists scalar(s) x (or y) such that $xP = Q$ (or $P = yQ$)).

Definition 1.7. A function $f : \mathbb{N} \rightarrow \mathbb{R}$ is said to be *polynomially bounded* if there exists $d \in \mathbb{N}$ and $r_0, r_1, \dots, r_d \in \mathbb{R}$ such that:

$$P(x) = r_0 + r_1x + r_2x^2 + \dots + r_dx^d \geq f(x) \quad \text{for all sufficiently large } x \quad (1.16)$$

or more standardly, if $\exists k \in \mathbb{N}$ s.t. $f(n) = O(g(n))$ where O is big-O notation and $g(n) = n^k$. I.e., $\exists M \in \mathbb{R}$ sufficiently large where $|f(x)| \leq Mg(x)$ for all $x \geq x_0$, for some x_0 .

A *polynomial-time algorithm* is one whose running time grows as n^k for some k , and is considered efficient. An *exponential-time algorithm* grows as k^n , for some $k > 1$, and is considered inefficient.

Definition 1.8. A function $\epsilon : \mathbb{N} \rightarrow [0, 1]$ is said to be *negligible* if for every polynomial $p(n)$, $\exists N \in \mathbb{N}$ s.t. $\epsilon(n) < \frac{1}{p(n)}$ for all $n > N$. In particular $\epsilon(n) \rightarrow 0$ as $n \rightarrow \infty$, but the converse fails: $\epsilon(n) = 1/n$ tends to 0 yet is not negligible, since $\epsilon(n) > \frac{1}{2^n}$ for all n .

Definition 1.9. A computational problem is considered *hard* if no polynomial-time algorithm (see Definition 1.7) exists that solves it with non-negligible probability (see Definition 1.8).

Definition 1.10. The discrete log problem for elliptic curve points (ECDLP) is *hard* if for any probabilistic polynomial-time (PPT) algorithm $\mathcal{A}$, the probability that $\mathcal{A}$ succeeds is negligible (see Definition 1.8):

$$\Pr[\mathcal{A}(G, kG) = k] \leq \epsilon(n) \quad (1.17)$$

Or more plainly, the probability that algorithm $\mathcal{A}$ correctly finds k when given G and kG is at most $\epsilon(n)$, which is negligible (very small).

More formally, for some security parameter λ , e.g. where λ is the bit-length of the curve order ($|E(\mathbb{F}_p)|$), then

$$\Pr_{\{G, k, r\}}[\mathcal{A}(G, Q = kG; r) = k] \leq \epsilon(\lambda) \quad (1.18)$$

where the probability is over the random choice of generator G and scalar $k \in \{0, 1, \dots, n-1\}$, and over the randomness r used by $\mathcal{A}$.

Definition 1.11 (Decisional Diffie-Hellman). In a cyclic group G of order q with generator g , the *Decisional Diffie-Hellman (DDH)* assumption states that, given g^a and g^b (both known), there is no efficient way to correlate or distinguish g^{ab} from a random element of G . Formally, the distributions (g^a, g^b, g^{ab}) and (g^a, g^b, g^c) are computationally indistinguishable when a, b, c are chosen uniformly at random from $\mathbb{Z}_q$.

Remark 1.6. In pairing-friendly groups (Section 5) DDH is easy: given (g^a, g^b, g^c) , one checks $e(g^a, g^b) \stackrel{?}{=} e(g, g^c)$. Pairing-based protocols therefore rest on other assumptions.

Theorem 1.7 (Hasse). Hasse’s theorem bounds the curve order for an equation E using the field order p :

$$p + 1 - 2\sqrt{p} \leq |E(\mathbb{F}_p)| \leq p + 1 + 2\sqrt{p} \quad (1.19)$$

Thus, we have outlined

1. the (algebraic) properties of an elliptic curve
2. the discrete log problem for elliptic curve points (ECDLP)
3. why ECDLP thwarts an adversary (or algorithm $\mathcal{A}$) running in polynomial time

The hardness of ECDLP (Definition 1.10) means that given generator G and point $Q = kG$, finding the scalar k is computationally infeasible for any polynomial-time algorithm (Definition 1.7). This property enables secure cryptography: an adversary cannot derive private keys from public keys, making elliptic curve cryptography secure for digital signatures (ECDSA) and key exchange (ECDH). The security parameter λ (bit-length of curve order) determines the security level—larger λ provides more security but requires more computation.

2 Rational Sums

In later protocol steps we often manipulate expressions that contain *division* (i.e., multiplicative inverses in $\mathbb{Z}_n$), but the elliptic curve itself only provides group operations on points. This section explains how to interpret such “rational scalars” via modular inversion and then map them through the scalar-to-point homomorphism φ . As a result, identities involving $\frac{a}{b}$ become ordinary elliptic-curve point equalities that can be checked using only point addition.

Despite not having division defined on elliptic curve points, we can extend the idea of rationals on the curve by taking mappings of $q \in \mathbb{Z}_n$, if $q = \frac{a}{b}$ with $\gcd(b, n) = 1$ then $\varphi(q) \in \langle G \rangle \subseteq E(\mathbb{F}_p)$, i.e., we get

$$\varphi(q) = \varphi(ab^{-1}) = (ab^{-1})G \quad (2.1)$$

where $b^{-1} = t$ such that $bt \equiv 1 \pmod{n}$. E.g., take $\frac{3}{7} \in \mathbb{Z}_{11}$, $\varphi(3 \times 7^{-1}) = \varphi(3 \times 8) = 2G$, as $7 \times 8 = 56 \equiv 1 \pmod{11}$.

Proposition 2.1. For $q_i, q_j \in \mathbb{Z}_n$, with $q_i = \frac{a_i}{b_i}$ or, $q_i = a_i b_i^{-1} \pmod{n}$ where $\gcd(b_i, n) = 1$, and $P_i = q_i G$, G generator. Similar for scalar q_j and point P_j . Rational scalars can be used to generate elliptic curve points while preserving the homomorphism. Specifically:

1. $P_i + P_j \in E(\mathbb{F}_p)$
2. and that this sum is unique

Proof. For the first point, by the isomorphism φ from Theorem 1.4, we have:

$$P_i + P_j = q_i G + q_j G = \varphi(q_i) + \varphi(q_j) = \varphi(q_i + q_j) = (q_i + q_j)G = \tilde{q}G \in E(\mathbb{F}_p) \quad (2.2)$$

The sum is unique because point addition is a function (Remark 1.3): each pair of points has exactly one sum. $\square$

Theorem 2.2 (Fermat's Little Theorem). Fermat's little theorem (see Appendix A) states that if p is prime, then for every $a \in \mathbb{Z}$ it holds that:

$$a^p \equiv a \pmod{p} \quad (2.3)$$

Corollary 2.3. If n is prime and $a \not\equiv 0 \pmod{n}$, then in calculating the multiplicative inverse one may use the identity

$$a^{n-2} \equiv a^{-1} \pmod{n}. \quad (2.4)$$

This follows from Fermat's Little Theorem applied in the prime field $\mathbb{Z}_n$.

3 ECP Matrix

Once scalars (including rational ones) can be mapped to curve points while preserving the homomorphism, the next step is to handle systems of many linear relations. The ECP Matrix section shows how matrix-vector multiplication over scalars corresponds to the same linear combination over the associated curve points. This is the algebraic reason matrix-based constraint checks can be reduced to point-addition computations.

To solve general systems of elliptic curve point equations we turn to matrices and vectors. We desire to show

$$M_{n \times n} \vec{s} = \vec{o}G \quad (3.1)$$

where M is a matrix of scalars, $\vec{s} = [s_1, s_2, \dots, s_n]$ with entries $s_i \in \langle G \rangle$, scalar weights $\vec{o} = [o_1, o_2, \dots, o_n]$ for generator point G .

Proposition 3.1. Matrix-vector multiplication preserves the homomorphism φ from Theorem 1.4. That is, for a matrix M of scalars and a vector $\vec{s}$ of elliptic curve points, the homomorphism property extends to linear combinations.

Example 3.2. E.g. for points $P, Q \in E(\mathbb{F}_p)$ generated respectively by scalars p, q , and a 2×2 matrix, we have:

$$\begin{bmatrix} 2 & 3 \\ 5 & 7 \end{bmatrix} \begin{bmatrix} P \\ Q \end{bmatrix} = \begin{bmatrix} 2P + 3Q \\ 5P + 7Q \end{bmatrix} = \begin{bmatrix} 2pG + 3qG \\ 5pG + 7qG \end{bmatrix} \quad (3.2)$$

where as φ is a homomorphism (by Theorem 1.4) we get:

$$2pG + 3qG = (2p + 3q)G \quad (3.3)$$

and similar for the second row so

$$\begin{bmatrix} 2 & 3 \\ 5 & 7 \end{bmatrix} \begin{bmatrix} P \\ Q \end{bmatrix} = \begin{bmatrix} 2p + 3q \\ 5p + 7q \end{bmatrix} G \quad (3.4)$$

an illustration that this homomorphism holds for the given systems of linear equations.

4 ECDSA

4.1 Overview

The Elliptic Curve Digital Signature Algorithm (ECDSA) is a digital signature scheme that uses the group law of an elliptic curve to produce compact signatures on messages.⁴ A signer holding a private key d can generate a signature (r, s) that can be verified by anyone using the corresponding public key $Q = dG$. In Ethereum-style encodings, one often appends a recovery bit v ; this is tied to public-key recovery and indicates which of the two possible y -coordinates corresponds to the point $R = [k]G$, typically encoded via the parity of that y -coordinate rather than a literal “positive/negative” sign. The scheme’s security is based on the hardness of the elliptic curve discrete logarithm problem (ECDLP): from public data G and $Q = dG$, an attacker cannot efficiently recover d .

4.2 ECDSA signature generation on secp256k1

Prover and Verifier agree on

1. a curve (secp256k1)
2. a base field order p
3. generator point G
4. a scalar field order n (curve order) — derived from 1. and 2.

Signing.

1. Choose private key $d \in \{1, \dots, n-1\}$
2. Compute public key: $Q = [d]G$
3. Hash message: $h = \text{hash}(m)$
4. Choose random nonce $k \in \{1, \dots, n-1\}$
5. Compute $R = [k]G$ and set $r \equiv x(R) \pmod{n}$; if $r = 0$, resample k
6. Compute $s \equiv k^{-1}(h + rd) \pmod{n}$; if $s = 0$, resample k
7. Send signature (r, s) ; in recoverable encodings one additionally sends v

Verification.

1. Check $r, s \in \{1, \dots, n-1\}$; reject otherwise
2. Hash message: $h = \text{hash}(m)$
3. Compute $w = s^{-1} \pmod{n}$, $u_1 = hw \pmod{n}$, and $u_2 = rw \pmod{n}$
4. Compute $R' = [u_1]G + [u_2]Q$
5. Accept if $R' \neq \mathcal{O}$ and $r \equiv x(R') \pmod{n}$

For an honest signature, verification succeeds because

$$u_1G + u_2Q = (hw)G + (rw)dG = w(h + rd)G = s^{-1}(h + rd)G = kG = R,$$

so $x(R') = x(R) \equiv r \pmod{n}$.

⁴A simpler signature scheme predating ECDSA is due to Claus-Peter Schnorr: signing is one linear equation in the nonce and private key, with no modular inversion. Schnorr patented the scheme (US 4,995,082, expired 2008), so it saw little adoption, and standardisation and implementation research concentrated on the unpatented (EC)DSA instead. Bitcoin adopted Schnorr signatures in 2021 (BIP-340, Taproot).

4.3 ECDSA FAQs

1. A random scalar k is chosen, generating point $R = [k]G$ on the curve. The value r is the x -coordinate of R reduced modulo n , and the value $s \equiv k^{-1}(h + rd) \pmod{n}$, where h is the message hash and d is the private key. In recoverable encodings, v stores auxiliary information for recovering the public key from (r, s) . The nonce k must never be reused: two signatures with the same k give two linear equations $s_i \equiv k^{-1}(h_i + rd) \pmod{n}$ in the two unknowns k and d , from which the private key d is recovered.
2. The verifier independently hashes the message to prevent signature forgery. If pre-hashed messages were accepted, an attacker could construct $R = aG + bQ$ for chosen scalars a, b , allowing them to forge signatures without knowing the private key. Explicitly: set $r = x(R) \pmod{n}$, $s = rb^{-1}$, and claim the “hash” $h = arb^{-1}$; then $u_1 = hs^{-1} = a$ and $u_2 = rs^{-1} = b$, so $R' = aG + bQ = R$ and (r, s) verifies.

For more info on design choices see this article.

5 Bilinear Pairings

Remark 5.1 (Section Disclosure). This section is dense with mathematical notation and definitions, and requires an elementary knowledge of algebraic structures. It can be skipped after reading *Overview 5.1* below.

5.1 Overview

Bilinear pairings are the canonical architecture for establishing the correctness of many cryptographic implementations, including Groth16.

5.2 Basic Theory & Definitions

Definition 5.1 (Field extension). If K is a field containing the subfield F , then K is an extension of F , denoted K/F .

Definition 5.2 (Algebraic). For a field F and extension K/F , an element $k \in K$ is *algebraic* over F if k is the root of some nonzero polynomial $f(x) \in F[x]$, and K/F is an *algebraic extension* if every $k \in K$ is algebraic over F .

If $F \subseteq L \subseteq K$ and $k \in K$ is algebraic over F , then k is also algebraic over L , because any polynomial in $F[x]$ is also a polynomial in $L[x]$.

Definition 5.3 (Algebraic Closure). A field F is *algebraically closed* if every polynomial with coefficients in F has a root in F ⁵.

Consequently, the *algebraic closure* of F , denoted $\overline{F}$, is algebraically closed⁶.

Definition 5.4 (Characteristic). For a field F , its *characteristic* denoted $\text{ch}(F)$,

$$\text{ch}(F) = \begin{cases} p, & \text{if } p \in \mathbb{Z}_{>0} \text{ exists s.t. } p \cdot 1_F = 0, \text{ then smallest such } p, \\ 0, & \text{if such a } p \text{ does not exist.} \end{cases}$$

⁵See Abstract Algebra chapter 13.4 for a rigorous treatment of this topic. For example, $x^2 - 2 \in \mathbb{Q}[x]$ has roots $\pm\sqrt{2}$ in $\mathbb{R} \setminus \mathbb{Q}$, so $\mathbb{Q}$ is not algebraically closed. Likewise, $x^2 + 1 \in \mathbb{R}[x]$ has roots $\pm i$ in $\mathbb{C} \setminus \mathbb{R}$, so $\mathbb{R}$ is not algebraically closed. By contrast, π is not an example of an algebraic root over $\mathbb{Q}$, because $x - \pi$ is not a polynomial in $\mathbb{Q}[x]$.

⁶See Abstract Algebra, chapter 13.4, proposition 29.

Definition 5.5 (r -torsion on curve). For an elliptic curve E over K , the r -torsion subgroup is

$$E[r] = \{P \in E(\overline{K}) : [r]P = \mathcal{O}\}.$$

Thus $E[r]$ consists of points whose order divides r . In the cryptographic setting where r is a large prime divisor of $|E(\mathbb{F}_p)|$, the prime-order subgroup used for security is generated by some G with $|G| = r$, namely

$$\langle G \rangle = \{\mathcal{O}, G, 2G, \dots, (r-1)G\}.$$

Remark 5.2 (Torsion). For a ring R , an element m of R -module M is called a *torsion element* if $rm = 0$ for nonzero $r \in R$. We define the set of torsion elements as

$$\text{Tor}(M) = \{m \in M \mid rm = 0_R \text{ for some nonzero } r \in R\}$$

Definition 5.6 (r -th roots of unity). The group of r -th roots of unity is

$$\mu_r = \{z \in \overline{K}^\times : z^r = 1\}. \quad (5.1)$$

Definition 5.7 (Embedding degree). Let $E/\mathbb{F}_p$ be an elliptic curve and let r be a prime dividing $|E(\mathbb{F}_p)|$. The *embedding degree* of E with respect to r is the smallest positive integer k such that

$$\mu_r \subseteq \mathbb{F}_{p^k}^\times. \quad (5.2)$$

Equivalently, k is the smallest positive integer for which $r \mid p^k - 1$.

Remark 5.3 (Linear Map). To contrast with bilinearity below, the definition of a linear map is presented here.

For vector spaces V and W over a field $\mathbb{F}$, a mapping $f : V \rightarrow W$ is linear, for elements $\vec{a}, \vec{b} \in V$ and $c \in \mathbb{F}$:

1. $f(\vec{a} + \vec{b}) = f(\vec{a}) + f(\vec{b})$
2. $f(c\vec{b}) = cf(\vec{b})$

These structure preserving maps are also known as a *homomorphism*. Every linear map is an additive group homomorphism (see Def. 1.5), but not every group homomorphism is a linear map.

Definition 5.8 (Bilinear Map of Abelian Groups). Let $(M, +)$ and $(N, +)$ be abelian groups and let $(R, *)$ be a group (often abelian, written multiplicatively). A map $f : M \times N \rightarrow R$ is *bilinear* if for all $x, y \in M$ and $z, w \in N$,

$$\begin{aligned} f(x + y, z) &= f(x, z) * f(y, z), \\ f(x, z + w) &= f(x, z) * f(x, w). \end{aligned}$$

We will examine bilinear maps, or *bilinear pairings*⁷, which take two elliptic-curve group (ECG) elements, from *two* cyclic groups G_1 and G_2 (relaxing the definition) of prime order that returns an element of a multiplicative target group G_T , denoted often by the mapping:

$$e : G_1 \times G_2 \rightarrow G_T \quad (5.3)$$

Consider an extension $\mathbb{F}_{p^k}/\mathbb{F}_p$ for p prime and $k \geq 1$. For pairing-based cryptography, groups G_1 and G_2 are defined in $E(\mathbb{F}_{p^k})$ and G_T in $\mathbb{F}_{p^k}^\times$ where E is as shown in equation (1.1). Given the bilinearity of e , we have for points $P, P' \in G_1$ and $Q, Q' \in G_2$,

⁷In a more algebraic formulation, bilinear maps are exactly the maps that factor through a tensor product via its universal property. This is why tensor products provide the natural abstract setting for pairings, even though the present section keeps the discussion concrete.

$$\begin{aligned} e(P + P', Q) &= e(P, Q) * e(P', Q) \\ e(P, Q + Q') &= e(P, Q) * e(P, Q') \end{aligned} \tag{5.4}$$

which, for scalars $a, b \in \mathbb{Z}$ and multiplication as described in equations (1.13), yields

$$e([a]P, [b]Q) = e(P, [b]Q)^a = e(P, Q)^{ab} = e([b]P, [a]Q) \tag{5.5}$$

Thus, for scalars a, b in a commutative ring (here $\mathbb{Z}$, or $\mathbb{Z}/r\mathbb{Z}$ for points of order dividing r), the pairing e is *symmetric* in the exponents (a, b) in the sense above. For *curve* inputs the situation is different: a cryptographic pairing is a map $e : G_1 \times G_2 \rightarrow G_T$, so for $P \in G_1$ and $Q \in G_2$ with $G_1 \neq G_2$ the swapped pair (Q, P) usually does not lie in $G_1 \times G_2$, and there is no reason to expect $e(P, Q) = e(Q, P)$. Even when both values can be defined (e.g. the Weil pairing on $E[r] \times E[r]$), they need not agree.

Remark 5.4 (Target group and embedding degree). The set $\mu_r = \{z \in \mathbb{F}_{p^k}^\times : z^r = 1\}$ is the kernel of the homomorphism $z \mapsto z^r$ on $\mathbb{F}_{p^k}^\times$, and is therefore a subgroup⁸. However, $|\mu_r|$ need not equal r : without the condition $r \mid p^k - 1$, the subgroup may contain fewer than r elements—in the extreme, only $\{1\}$. The *embedding degree* k is the smallest positive integer such that $r \mid p^k - 1$. Since $\mathbb{F}_{p^k}^\times$ is cyclic of order $p^k - 1$ and $r \mid p^k - 1$, the group has a unique subgroup of order r ; this subgroup consists of exactly r distinct roots of $x^r - 1$, so $|\mu_r| = r$.

In pairing-based cryptography, the target group is $G_T = \mu_r \subset \mathbb{F}_{p^k}^\times$. To see why the pairing lands in μ_r : since $P \in G_1$ has order r , bilinearity gives $e(P, Q)^r = e([r]P, Q) = e(\mathcal{O}, Q) = 1$, so every output is an r -th root of unity and the image is a subgroup of μ_r . Since r is prime, this subgroup is either $\{1\}$ or all of μ_r ; non-degeneracy (Definition 5.17) rules out the trivial case, so $G_T = \mu_r$.

5.3 Divisors and Rational Functions for Pairings

To connect with the later cryptographic applications, keep in mind that constructions such as Groth16 work with designated source groups G_1 and G_2 and a pairing $e : G_1 \times G_2 \rightarrow G_T$. The divisor-theoretic discussion below, however, is presented first in the setting of the Weil pairing on the full r -torsion $E[r]$. In that Weil-pairing setting one represents points by degree-zero divisors such as $(P) - (\mathcal{O})$, and one must choose equivalent representatives with disjoint supports so that function evaluation is well defined.

Definition 5.9 (Divisor (Group)). For a curve C over K , the *divisor group*, denoted $\text{Div}_{\overline{K}}(C)$, is the free⁹ abelian group generated by the points of C . From this group, we denote a selection of elements as a *divisor* $D \in \text{Div}(C)$ as a formal sum

$$D = \sum_{P \in C} n_P(P)$$

where $n_P \in \mathbb{Z}$ is the *multiplicity* of point P in the divisor D and $n_P = 0$ for all but finitely many points $P \in C$. Here (P) denotes the divisor with coefficient 1 at the point P and coefficient 0 at all other points, so $\text{Div}(C)$ is the free abelian group generated by the symbols (P) and coefficient-wise addition. Equivalently, its elements are finite formal sums $\sum_P n_P(P)$ with integer coefficients. Thus $D = \sum_{P \in C} n_P(P)$ is a formal sum in the divisor group, not an elliptic-curve sum of points. In particular, $2(P) = (P) + (P)$ in $\text{Div}(C)$, which is distinct from the elliptic-curve multiple $[2]P$. The word “free” here means that the points furnish independent $\mathbb{Z}$ -generators for a free abelian group; this is separate from the tensor-product formalism, although one may later view $\text{Div}(C)$ as a free $\mathbb{Z}$ -module.

⁸The kernel of any group homomorphism $\varphi : G \rightarrow H$ is a subgroup of G : the identity maps to the identity, the product of two kernel elements maps to the identity, and the inverse of a kernel element maps to the identity.

⁹See Abstract Algebra chapter 6.3 for a light treatment of this topic.

Definition 5.10. The *degree* of a divisor D is

$$\text{Deg}(D) = \sum_{P \in C} n_P$$

Definition 5.11 (Support of Divisor). The support of a divisor D ,

$$\text{supp}(D) = \{P \in C(\overline{K}) : n_P \neq 0\}$$

or, specifically to our context, $\text{supp}(D) = \{P \in E(\overline{\mathbb{F}_p}) : n_P \neq 0\}$.

Example 5.5. For a curve E over a field $\mathbb{F}_p$, let points $P, Q \in E(\overline{\mathbb{F}_p})$ and divisors D_1, D_2 , with $D_1 = 3(P) + 4(Q)$ and $D_2 = -2(Q)$, then

$$\text{Deg}(D_1) = 3 + 4 = 7, \text{ Deg}(D_2) = -2,$$

$$D_1 + D_2 = 3(P) + 2(Q), \text{ and } \text{Deg}(D_1 + D_2) = 3 + 2 = 5,$$

with

$$\text{supp}(D_1) = \{P, Q\}, \text{ supp}(D_2) = \{Q\}, \text{ supp}(D_1 + D_2) = \{P, Q\}.$$

Definition 5.12 (Order of a function at a point). The order of a function f at a point P , denoted $\text{ord}_P(f)$ ¹⁰, is the multiplicity of f at P on the curve. Writing $m \in \mathbb{Z}_{>0}$ for multiplicities,

$$\text{ord}_P(f) = \begin{cases} m, & \text{if } f \text{ has a zero of multiplicity } m \text{ at } P, \\ -m, & \text{if } f \text{ has a pole of multiplicity } m \text{ at } P, \\ 0, & \text{if } f \text{ is regular and nonvanishing at } P. \end{cases} \quad (5.6)$$

An order describes how a function behaves at the actual curve point P : for example, $\text{ord}_P(f) = -2$ means that near P the function behaves like $1/u^2$ up to a nonzero regular factor, where u is a local parameter with $u(P) = 0$, i.e., mapping $u : E \rightarrow \mathbb{F}_p$ with $\deg(u) = 1(\mathcal{O})$, where $u \in K(E)$ (see Remark B.1). For what “function on a curve” means, and for zeros, poles, regularity, and multiplicity in local coordinates, see Appendix B.

Definition 5.13 (Divisor of a function). Define the divisors of a function as,

$$\text{div}(f) = \sum_{P \in C} \text{ord}_P(f)(P)$$

This is an element of the same divisor group $\text{Div}(C)$ defined above: the symbols (P) remain the formal generators, while $\text{div}(f)$ records the integer coefficients $\text{ord}_P(f)$ attached to them. Thus $\text{ord}_P(f)$ is local data at the point P , and $\text{div}(f)$ packages those local orders into one global divisor.

Additionally, a divisor $D \in \text{Div}(C)$ is *principal* if there exists an $f \in \overline{K}(C)^\times$ such that $\text{div}(f) = D$. Equivalently, the divisors of functions are exactly the principal divisors.

Remark 5.6 (Summary of the basic objects). A point P is a point on the curve E , usually over the base field or its algebraic closure, and one should also remember the point at infinity $\mathcal{O}$. The symbol (P) denotes the divisor with coefficient 1 at P and coefficient 0 elsewhere. For a nonzero rational function f , the divisor $\text{div}(f)$ is the divisor attached to f via its zeros and poles. A divisor is *principal* precisely when it is of the form $\text{div}(f)$ for some nonzero rational function f .

¹⁰See Silverman, *The Arithmetic of Elliptic Curves*, Ch. II, §1 (local rings $\mathcal{O}_{C,P}$, maximal ideals, and valuations) and §3 (principal divisors and $\text{div}(f)$).

Example 5.7. If a rational function f on E satisfies

$$\operatorname{div}(f) = 3(P) + 4(Q) - 7(\mathcal{O}),$$

then f has a zero of order 3 at P , a zero of order 4 at Q , and a pole of order 7 at $\mathcal{O}$. The degree is

$$3 + 4 - 7 = 0,$$

illustrating that principal divisors have degree zero.

Theorem 5.8 (Principal divisors have degree zero). If C is a smooth projective curve and $f \in K(C)^\times$ is a nonzero rational function, then

$$\operatorname{Deg}(\operatorname{div}(f)) = 0.$$

Equivalently, the total multiplicity of the zeros of f equals the total multiplicity of its poles.

Proof sketch. If f is constant, then $\operatorname{div}(f) = 0$, then the conclusion is immediate.

If f is nonconstant, then it defines a morphism (rational map regular at every point) $f : C \rightarrow \mathbb{P}^1$, where $\mathbb{P}^1$ is the projective line. The zeros of f are exactly the points of C mapping to $0 \in \mathbb{P}^1$, and the poles of f are exactly the points mapping to $\infty \in \mathbb{P}^1$, each counted with multiplicity. Here the fiber above a point $a \in \mathbb{P}^1$ means the preimage $f^{-1}(a)$. By the defining property of the degree of a nonconstant morphism of smooth projective curves, for a fixed chosen value $a \in \mathbb{P}^1$ one may write

$$f^{-1}(a) = \{P_1, \dots, P_t\},$$

where each P_i is a point of C . One writes $e_f(P_i) \in \mathbb{Z}_{\geq 0}$ for the multiplicity of the map f at the point P_i in this fiber; this is the local multiplicity of the morphism at P_i . Concretely, in the fiber above 0 it agrees with the zero order of f at P_i , so

$$\operatorname{ord}_{P_i}(f) = e_f(P_i)$$

for points with $f(P_i) = 0$. In the fiber above ∞ , the corresponding pole order is

$$-\operatorname{ord}_{P_i}(f) = e_f(P_i).$$

These multiplicities need not be equal. Their sum is

$$e_f(P_1) + \dots + e_f(P_t) = \deg(f).$$

This statement is applied separately to the fiber above 0 and to the fiber above ∞ . In both cases the fiber multiplicities $e_f(P_i)$ are positive integers whose sum is $\deg(f)$. For points above 0, these multiplicities are the zero orders $\operatorname{ord}_{P_i}(f) > 0$; for points above ∞ , they are the pole orders $-\operatorname{ord}_{P_i}(f) > 0$. Thus the total zero multiplicity and the total pole multiplicity are both $\deg(f)$, so

$$\operatorname{Deg}(\operatorname{div}(f)) = 0.$$

A full proof can be found in Silverman, *The Arithmetic of Elliptic Curves*, Ch. II, §3, Proposition 3.1. (b), where principal divisors on elliptic curves are treated in the standard divisor formalism. $\square$

Example 5.9. A simple model is obtained by taking $C = \mathbb{P}^1$ and considering the degree-2 map

$$\begin{aligned} f : \mathbb{P}^1 &\rightarrow \mathbb{P}^1 \\ z &\mapsto z^2. \end{aligned}$$

This is relevant because the theorem applies to every smooth projective curve C ; here we are simply illustrating it in the special case $C = \mathbb{P}^1$. Over an algebraically closed field, a generic

target value $a \in \mathbb{P}^1$ with $a \neq 0, \infty$ has two distinct preimages, namely $\sqrt{a}$ and $-\sqrt{a}$. So a generic fiber has two points, each with multiplicity 1, and hence total multiplicity 2. The fiber above 1 is the concrete example $\{1, -1\}$, again with total multiplicity 2. The fiber above 0 is $\{0\}$, but now the point 0 occurs with multiplicity 2 for the map $z \mapsto z^2$, so the total multiplicity is still 2. Likewise, the fiber above ∞ is $\{\infty\}$ with multiplicity 2. Thus the degree is not counting only distinct points in a fiber; it counts points with multiplicity. Some fibers have two distinct points, while special fibers such as $\{0\}$ and $\{\infty\}$ have only one distinct point, but still total multiplicity 2.

Remark 5.10 (Principal divisors inside degree-zero divisors).

$$\text{Prin}(C) \subset \text{Div}^0(C) = \{D \in \text{Div}(C) : \text{Deg}(D) = 0\} \subset \text{Div}(C).$$

Thus principal divisors form a subgroup of the degree-zero divisors.

A term such as $-7(\mathcal{O})$ means coefficient -7 at the point at infinity; it is not “seven points at infinity,” but rather that in the divisor the point $\mathcal{O}$ appears with coefficient -7 . When it occurs in $\text{div}(f)$, it records a pole of order 7 at $\mathcal{O}$. The advantage of this notation is that zeros and poles at finite points and at infinity are all encoded uniformly in one divisor.

Remark 5.11 (Algebra of principal divisors). For nonzero rational functions f, g on C ,

$$\text{div}(fg) = \text{div}(f) + \text{div}(g), \quad \text{div}(f/g) = \text{div}(f) - \text{div}(g).$$

If $c \in K^\times$ is constant, then $\text{div}(c) = 0$. In particular, if $f = g$, then $\text{div}(f/g) = \text{div}(1) = 0$.

Definition 5.14 (Equivalent divisors). Two divisors D_1 and D_2 are said to be *equivalent*, written $D_1 \sim D_2$, if their difference is principal:

$$D_1 - D_2 = \text{div}(f)$$

for some nonzero rational function f .

Remark 5.12 (Intuition for divisor equivalence). Saying $D_1 \sim D_2$ means that $D_1 - D_2 = \text{div}(f)$ for some nonzero rational function f . If another function g satisfies $\text{div}(g) = \text{div}(f)$, then

$$\text{div}(g/f) = \text{div}(g) - \text{div}(f) = 0.$$

Thus g/f has no zeros and no poles, so on a smooth projective curve it must be a nonzero constant. Hence $g = cf$ for some $c \in K^\times$. Therefore all functions witnessing the same principal divisor have the same zeros and poles with the same multiplicities; they differ only by multiplication by a nonzero constant. What matters is therefore the principal divisor $\text{div}(f)$, not the particular scalar multiple chosen to witness it.

Definition 5.15 (Evaluation of a function at a divisor). For a rational function $f \in K(C)^\times$ and a divisor $D = \sum_P n_P(P)$, if $\text{supp}(\text{div}(f)) \cap \text{supp}(D) = \emptyset$, that is, the supports of $\text{div}(f)$ and D are disjoint, then evaluating f at D is:

$$f(D) = \prod_{P \in C} f(P)^{n_P}$$

In the pairing setting one usually specializes this to the case $C = E$ and $f \in K(E)^\times$ for a suitable base field K such as $\mathbb{F}_p$ or $\mathbb{F}_{p^k}$.

Remark 5.13 (Why disjoint support is required). The disjointness condition ensures that every factor $f(P)^{n_P}$ is well defined. If P lay in $\text{supp}(\text{div}(f))$, then f would have either a zero or a pole at P , so $f(P)$ would fail to be a nonzero finite value and the product above would break down. Disjoint support is therefore a well-definedness condition, not an extra algebraic structure.

Theorem 5.14 (Weil’s reciprocity law). For non-zero rational functions f, g on a curve C , if $\text{supp}(\text{div}(f)) \cap \text{supp}(\text{div}(g)) = \emptyset$ then

$$f(\text{div}(g)) = g(\text{div}(f))$$

Proposition 5.15 (The functions $f_{m,P}$). For $P \in E$ and an integer $m \geq 1$, there exists a rational function $f_{m,P}$ whose divisor is

$$\text{div}(f_{m,P}) = m(P) - ([m]P) - (m-1)(\mathcal{O}). \quad (5.7)$$

In particular, if $P \in E[r]$, then

$$\text{div}(f_{r,P}) = r(P) - r(\mathcal{O}). \quad (5.8)$$

Here $f_{m,P}$ is simply a rational function attached to the pair (m, P) , where m is a positive integer and P is a point of E ; the notation does not mean “evaluate a function at P ”. These functions are the basic ingredients in the definitions of the Weil and Tate pairings, because divisor notation records exactly the zeros and poles that determine them.

Remark 5.16 (Choosing disjoint representatives). For the divisor classes represented by $(P) - (\mathcal{O})$ and $(Q) - (\mathcal{O})$, one may choose equivalent degree-zero divisors D_P and D_Q with disjoint supports, so the evaluations appearing below are well defined.¹¹

5.4 Weil and Tate Pairings

Let r be prime with $r \neq \text{ch}(K)$. Then one has

$$E[r] \cong \mathbb{Z}_r \times \mathbb{Z}_r^{12}. \quad (5.9)$$

Definition 5.16 (Weil pairing). Let $P, Q \in E[r]$. Choose degree-zero divisors D_P and D_Q with disjoint supports such that

$$D_P \sim (P) - (\mathcal{O}), \quad D_Q \sim (Q) - (\mathcal{O}).$$

Choose rational functions f and g such that

$$\text{div}(f) = rD_P, \quad \text{div}(g) = rD_Q.$$

Here g is any rational function with divisor rD_Q (equivalently, encoding the Q -side data up to a nonzero scalar); f encodes the P -side data. The *Weil pairing* is

$$w_r(P, Q) = \frac{f(D_Q)}{g(D_P)} \in \mu_r. \quad (5.10)$$

Expositions often compress the Q -side data into a single rational function g and write a ratio of *point evaluations*

$$w_r(P, Q) = \frac{g(X+P)}{g(X)},$$

where $X \in E$ is an auxiliary point (or a formal generic in algebraic treatments) chosen disjoint from the relevant supports, and $X+P$ is addition on E . Equation (5.10) is the divisor-theoretic presentation used here. In Silverman, Ch. III, §8, one first fixes $T \in E[m]$, chooses a rational function g attached to that T -side data, and then defines the Weil pairing by

$$e_m(S, T) = \frac{g(X+S)}{g(X)},$$

¹¹This is standard in the divisor-theoretic construction of the Weil pairing; see Silverman, *The Arithmetic of Elliptic Curves*, Ch. III, §8, and Costello, *Pairings for beginners*, introductory sections on the Weil pairing.

¹²For a proof, see Silverman, *The Arithmetic of Elliptic Curves*, Ch. III, Cor. 6.4(b).

where X is chosen so that both values are defined and nonzero. The point is that translating X by the torsion point S changes g only by an m -th root of unity, so this ratio is independent of the chosen X . Silverman then gives, in Remark 8.5, an alternative formulation much closer in spirit to divisor evaluation.

Remark 5.17 (Why the Weil pairing lands in μ_r). The value of the Weil pairing is not a point of the elliptic-curve torsion subgroup $E[r]$; rather, it is an element of the multiplicative group μ_r of r -th roots of unity. Indeed, Weil reciprocity gives

$$f(\operatorname{div}(g)) = g(\operatorname{div}(f)).$$

Since $\operatorname{div}(g) = rD_Q$ and $\operatorname{div}(f) = rD_P$, this becomes

$$f(rD_Q) = g(rD_P).$$

By the definition of evaluating a function at a divisor,

$$f(D_Q)^r = g(D_P)^r.$$

Therefore

$$\left(\frac{f(D_Q)}{g(D_P)} \right)^r = 1.$$

Since $w_r(P, Q) = f(D_Q)/g(D_P)$, it follows that

$$w_r(P, Q)^r = 1,$$

so $w_r(P, Q) \in \mu_r$.

Theorem 5.18 (Well-definedness of the Weil pairing). The value

$$w_r(P, Q) = \frac{f(D_Q)}{g(D_P)}$$

is independent of the chosen equivalent divisors D_P, D_Q and of the chosen rational functions f, g satisfying $\operatorname{div}(f) = rD_P$ and $\operatorname{div}(g) = rD_Q$. Hence $w_r(P, Q)$ depends only on the points $P, Q \in E[r]$.¹³

Definition 5.17 (Non-degenerate pairing). A bilinear pairing $e : G_1 \times G_2 \rightarrow G_T$ is *non-degenerate* if for every nontrivial $P \in G_1$ there exists $Q \in G_2$ such that $e(P, Q) \neq 1$, and for every nontrivial $Q \in G_2$ there exists $P \in G_1$ such that $e(P, Q) \neq 1$. Equivalently, the induced homomorphisms $G_1 \rightarrow \operatorname{Hom}(G_2, G_T)$ and $G_2 \rightarrow \operatorname{Hom}(G_1, G_T)$ are injective.

Remark 5.19 (Homomorphisms, kernels, and non-degeneracy). Here *nontrivial* means “not equal to the identity element” of the relevant group, so for G_1 and G_2 it means a point different from $\mathcal{O}$, and for G_T it means a value different from 1. For a fixed $P \in G_1$, bilinearity in the second variable shows that

$$\begin{aligned} \varphi_P : G_2 &\rightarrow G_T \\ Q &\mapsto e(P, Q) \end{aligned}$$

is a group homomorphism. Likewise, for fixed $Q \in G_2$ one obtains a homomorphism $G_1 \rightarrow G_T$, $P \mapsto e(P, Q)$. Thus the pairing induces maps

$$\begin{aligned} \Phi : G_1 &\rightarrow \operatorname{Hom}(G_2, G_T) \\ P &\mapsto \varphi_P, \end{aligned}$$

and similarly on the other side. The kernel of Φ consists exactly of those $P \in G_1$ such that $e(P, Q) = 1$ for all $Q \in G_2$. Therefore non-degeneracy on the G_1 -side is equivalent to $\ker(\Phi) = \{\mathcal{O}\}$, which is equivalent to Φ being injective. The same argument applies on the G_2 -side.

¹³See Silverman, *The Arithmetic of Elliptic Curves*, Ch. III, §8 for the standard proof that the divisor-form definition is independent of all auxiliary choices. Costello, *Pairings for beginners*, gives the same conclusion in introductory pairing language.

Theorem 5.20 (Basic properties of the Weil pairing). The Weil pairing

$$w_r : E[r] \times E[r] \rightarrow \mu_r$$

is bilinear and non-degenerate. It is also alternating, so $w_r(P, P) = 1$ for all $P \in E[r]$, and therefore

$$w_r(P, Q) = w_r(Q, P)^{-1}.$$

These are the structural properties that make the Weil pairing useful in both the theory of elliptic curves and pairing-based cryptography.¹⁴

Definition 5.18 (Tate pairing). Let

$$rE(\mathbb{F}_{p^k}) = \{[r]R : R \in E(\mathbb{F}_{p^k})\}.$$

If $P \in E[r]$ and Q represents a class in $E(\mathbb{F}_{p^k})/rE(\mathbb{F}_{p^k})$, and if D_Q is a degree-zero divisor equivalent to $(Q) - (\mathcal{O})$ with support disjoint from $\text{supp}(\text{div}(f_{r,P}))$, then the *Tate pairing* is

$$t_r(P, Q) = f_{r,P}(D_Q), \quad (5.11)$$

viewed modulo r -th powers in $\mathbb{F}_{p^k}^\times$.

Remark 5.21 (What “represents a class” means). Saying that Q *represents a class* in $E(\mathbb{F}_{p^k})/rE(\mathbb{F}_{p^k})$ means that the real datum is the coset

$$Q + rE(\mathbb{F}_{p^k}) = \{Q + [r]R : R \in E(\mathbb{F}_{p^k})\},$$

and Q is simply one chosen point from that set. Any other point $Q' = Q + [r]R$ belongs to the same coset, so Q and Q' are merely different *representatives* of the same quotient class. Two points represent the same class exactly when their difference lies in $rE(\mathbb{F}_{p^k})$.

Remark 5.22 (Why the Tate pairing is quotient-valued). Here “quotient-valued” means the following. Suppose Q' is another representative of the same class as Q in $E(\mathbb{F}_{p^k})/rE(\mathbb{F}_{p^k})$. Then $Q' = Q + [r]R$ for some $R \in E(\mathbb{F}_{p^k})$, and the corresponding unreduced Tate values differ by an r -th power:

$$t_r(P, Q') = t_r(P, Q) \cdot u^r$$

for some $u \in \mathbb{F}_{p^k}^\times$. Equivalently,

$$\frac{t_r(P, Q')}{t_r(P, Q)} \in (\mathbb{F}_{p^k}^\times)^r.$$

So the unreduced Tate pairing is not a single well-defined element of $\mathbb{F}_{p^k}^\times$; it is only well-defined up to multiplication by an r -th power. This is exactly why its natural codomain is the quotient group $\mathbb{F}_{p^k}^\times / (\mathbb{F}_{p^k}^\times)^r$.¹⁵

Definition 5.19 (Reduced Tate pairing). The *reduced Tate pairing* is obtained by raising the Tate pairing to the power

$$\frac{p^k - 1}{r}, \quad (5.12)$$

¹⁴See Silverman, *The Arithmetic of Elliptic Curves*, Ch. III, §8 for the standard theorem and its consequences. Costello, *Pairings for beginners*, emphasizes bilinearity, non-degeneracy, and alternating behavior as the key pairing properties for cryptography.

¹⁵For the standard quotient-valued definition and the reduction step, see Costello, *Pairings for beginners*, sections on the Tate and reduced Tate pairings. Silverman also discusses the Tate pairing formalism in the divisor-theoretic setting.

so that its value lies in μ_r . Indeed, every element of the multiplicative group $\mathbb{F}_{p^k}^\times$ has order dividing $p^k - 1$ (by Lagrange's theorem, since $|\mathbb{F}_{p^k}^\times| = p^k - 1$), so if one multiplies an unreduced Tate value by an r -th power u^r , then

$$(u^r)^{(p^k-1)/r} = u^{p^k-1} = 1.$$

Thus exponentiating by $(p^k - 1)/r$ removes the r -th-power ambiguity from the quotient-valued Tate pairing. Moreover, if

$$v = t_r(P, Q)^{(p^k-1)/r},$$

then

$$v^r = t_r(P, Q)^{p^k-1} = 1,$$

so the reduced value really does lie in μ_r .

Remark 5.23 (Two parallel quotients). The Tate pairing involves two quotient structures. On the elliptic-curve side, Q and $Q' = Q + [r]R$ are different representatives of the same class in $E(\mathbb{F}_{p^k})/rE(\mathbb{F}_{p^k})$: we stop distinguishing points that differ by an r -multiple. On the field side, if one representative gives unreduced value x and another gives $x \cdot u^r$, then x and $x \cdot u^r$ are different representatives of the same class in $\mathbb{F}_{p^k}^\times/(\mathbb{F}_{p^k}^\times)^r$: we stop distinguishing field elements that differ by an r -th power. The reduction exponent $(p^k - 1)/r$ collapses this second quotient into μ_r , producing a single well-defined field element.

The cryptographic importance of pairings lies in the identity

$$e([a]P, [b]Q) = e(P, Q)^{ab},$$

which converts multiplication of hidden scalars into multiplication in the target group. This bilinearity is used concretely in Section 7.3, where the verifier checks a QAP relation through a pairing equation, and it is then refined further in Section 8 for the full Groth16 construction.

6 Zero-Knowledge Proof Foundations

6.1 Arithmetic Circuits

For the remainder of this section the finite-field arithmetic is performed over $\mathbb{F}_r$, where r is the prime order of the pairing source groups G_1 and G_2 (with $r \mid |E(\mathbb{F}_p)|$). The curve itself remains defined over $\mathbb{F}_p$; why the proof-system arithmetic uses $\mathbb{F}_r$ rather than $\mathbb{F}_p$ is explained in Remark 7.10, once the relevant constructions have been introduced.

Definition 6.1. An *arithmetic circuit* over a finite field $\mathbb{F}_r$ is a system of polynomial equations of the form

$$\{e_i(x_1, x_2, \dots, x_n) = 0\}_{i=1}^m \tag{6.1}$$

where each e_i is a polynomial expression using addition, multiplication, and equality over $\mathbb{F}_r$. A circuit is *satisfied* if there exists an assignment $(x_1, x_2, \dots, x_n) \in \mathbb{F}_r^n$ such that all equations $e_i = 0$ hold simultaneously.

Arithmetic circuits provide a convenient representation for NP computational problems (problems where a solution can be verified in polynomial time).¹⁶ Additionally, arithmetic circuits can operate directly over field elements making them more concise than Binary circuits which require binary decomposition.

¹⁶P is the class of problems solvable in polynomial time; NP those whose solutions are verifiable in polynomial time. A problem is *NP-complete* if it lies in NP and every NP problem reduces to it in polynomial time, so an efficient algorithm for one would solve all of NP. Whether $P = NP$ is open; cryptography operates on the assumption that finding solutions is strictly harder than verifying them.

For example, the constraint that $x \in \{7, 12\}$ can be expressed in different ways:

As an *arithmetic circuit*:

$$(x - 7)(x - 12) = 0$$

As a *binary circuit*, expressing x in binary as $x = x_3 \cdot 2^3 + x_2 \cdot 2^2 + x_1 \cdot 2^1 + x_0 \cdot 1$ where $x_i \in \{0, 1\}$, and using a selector $b \in \{0, 1\}$:

$$\begin{aligned} x_3(x_3 - 1) &= 0, & x_2(x_2 - 1) &= 0, & x_1(x_1 - 1) &= 0, & x_0(x_0 - 1) &= 0 \\ b(b - 1) &= 0 \\ x &= x_3 \cdot 8 + x_2 \cdot 4 + x_1 \cdot 2 + x_0 \cdot 1 \\ x &= 7 + 5b \end{aligned}$$

As a *boolean circuit*, we first express x in binary as above and then add boolean constraints. For this example, we have b_7, b_{12} combined with statements for AND ($z = x \cdot y$) and OR ($z = x + y - x \cdot y$) (NOT is expressed as $(y = 1 - x)$).

$$\begin{aligned} x_3(x_3 - 1) &= 0, & x_2(x_2 - 1) &= 0, & x_1(x_1 - 1) &= 0, & x_0(x_0 - 1) &= 0 \\ x &= x_3 \cdot 8 + x_2 \cdot 4 + x_1 \cdot 2 + x_0 \cdot 1 \\ \text{Let } b_7 &= (1 - x_3) \cdot x_2 \cdot x_1 \cdot x_0 \quad (\text{checks if } x = 7 \text{ which is } 0111 \text{ in binary}) \\ \text{Let } b_{12} &= x_3 \cdot x_2 \cdot (1 - x_1) \cdot (1 - x_0) \quad (\text{checks if } x = 12 \text{ which is } 1100 \text{ in binary}) \\ b_7 + b_{12} - b_7 \cdot b_{12} &= 1 \quad (\text{OR: at least one must be true}) \end{aligned}$$

For this example the arithmetic circuit representation is the most concise, requiring only a single polynomial constraint. Generally, arithmetic circuits are more concise, however, several operations are cheaper in Boolean form.

Arithmetic circuits can represent any problem in NP. This stems from the Cook-Levin Theorem in computational complexity theory, which shows that every NP problem reduces in polynomial time to Boolean circuit satisfiability; arithmetic circuits can in turn describe every boolean circuit (via the gate encodings above: AND $z = xy$, OR $z = x + y - xy$, NOT $y = 1 - x$).

We can express any computation as a system of polynomial equations over a finite field, where satisfiability of the system corresponds to the existence of a valid solution to the computational problem.

6.2 Rank-1 Constraint Systems

Definition 6.2. A *Rank-1 Constraint System* (R1CS) over $\mathbb{F}_r$ consists of three matrices $\mathbf{L}, \mathbf{R}, \mathbf{O} \in \mathbb{F}_r^{m \times n}$ and a *witness vector* $\mathbf{a} \in \mathbb{F}_r^n$ of the form $\mathbf{a} = [a_1, a_2, \dots, a_n]^T$, where the first component is always $a_1 = 1$. The R1CS is satisfied if and only if

$$(\mathbf{L}\mathbf{a}) \circ (\mathbf{R}\mathbf{a}) = \mathbf{O}\mathbf{a} \tag{6.2}$$

where $\circ$ denotes the Hadamard (element-wise) product. By convention, the first element is always 1 to enable representing constant terms and affine combinations in the constraints.

For each constraint $i \in \{1, 2, \dots, m\}$, we have:

$$(\mathbf{L}_i \cdot \mathbf{a}) \cdot (\mathbf{R}_i \cdot \mathbf{a}) = \mathbf{O}_i \cdot \mathbf{a} \tag{6.3}$$

where $\mathbf{L}_i, \mathbf{R}_i, \mathbf{O}_i$ denote the i -th rows of matrices $\mathbf{L}, \mathbf{R}, \mathbf{O}$ respectively, and the witness vector $\mathbf{a}$ contains all variables in the system.

All operations in an R1CS are performed modulo the subgroup-order prime r . That is, all arithmetic is done in the finite field $\mathbb{F}_r$, where addition and multiplication are taken modulo r .

To construct the matrices $\mathbf{L}, \mathbf{R}, \mathbf{O}$ from an arithmetic constraint, we proceed as follows. For a constraint of the form (left) $\times$ (right) = (output), the matrices encode which variables appear in each position:

- $\mathbf{L}$ encodes the variables on the left-hand side of the multiplication
- $\mathbf{R}$ encodes the variables on the right-hand side of the multiplication
- $\mathbf{O}$ encodes the result variable

Each matrix has m rows (one per constraint) and n columns (one per variable in the witness vector). The entry $\mathbf{L}_{i,j}$ is the coefficient of the j -th variable in the left-hand side of constraint i .

For example, consider the constraint $z = x \cdot y$ with witness vector $\mathbf{a} = [1, z, x, y]^T$. The R1CS matrices are:

$$\mathbf{L} = \begin{bmatrix} 0 & 0 & 1 & 0 \end{bmatrix}, \quad \mathbf{R} = \begin{bmatrix} 0 & 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{O} = \begin{bmatrix} 0 & 1 & 0 & 0 \end{bmatrix} \quad (6.4)$$

This encodes: $(\mathbf{L}\mathbf{a}) \cdot (\mathbf{R}\mathbf{a}) = x \cdot y = z = \mathbf{O}\mathbf{a}$.

For constraints with multiple multiplications (e.g., $z = x^2 \cdot y$), we introduce intermediate variables to ensure exactly one multiplication per constraint:

$$v_1 = x \cdot x \quad (6.5)$$

$$z = v_1 \cdot y \quad (6.6)$$

The witness vector becomes $\mathbf{a} = [1, z, x, y, v_1]^T$, and we create two constraints, each with exactly one multiplication.

If a constraint has no multiplications (e.g., $z = x + y$), we can introduce a dummy multiplication by multiplying by 1: $z \cdot 1 = (x + y) \cdot 1$, or restructure as $z = x + y$ with $z - x - y = 0$, which can be encoded using the constant 1 in the witness vector.

Remark 6.1 (Why R1CS). An R1CS sits at a precise boundary in expressiveness. A purely linear system over $\mathbb{F}_r$ is solvable in polynomial time by Gaussian elimination, so linear constraints alone encode no hard relation. Permitting one multiplication per constraint is the minimal non-linearity at which satisfiability becomes NP-complete: any arithmetic circuit flattens into such constraints, so any NP claim “I know a witness such that the computation accepts” becomes “I know a vector $\mathbf{a}$ satisfying the system”, with $\mathbf{a}$ acting as the execution trace of the computation. The same boundary is the maximum our tools can verify: the homomorphism φ (Theorem 1.4) verifies linear combinations of hidden scalars, and a pairing verifies exactly one product, as the output of $e : G_1 \times G_2 \rightarrow G_T$ cannot be used as input to another pairing. Rank-1 quadratic constraints are thus the least structure that captures NP and the most structure a pairing can check; one constraint maps to one pairing. The name *rank-1* records that each constraint is the quadratic form $\mathbf{a}^T (\mathbf{L}_i^T \mathbf{R}_i) \mathbf{a} = \mathbf{O}_i \cdot \mathbf{a}$, whose matrix $\mathbf{L}_i^T \mathbf{R}_i$, an outer product, has rank at most one. R1CS is therefore a general intermediate representation rather than an artifact of pairings: circuit compilers target it, and proof systems without pairings (e.g., Bulletproofs) consume it directly.

6.3 Lagrange Interpolation

Definition 6.3 (Kronecker delta). For integers $1 \leq i, j \leq n$, the *Kronecker delta* δ_{ij} is defined by

$$\delta_{ij} = \begin{cases} 1 & \text{if } i = j, \\ 0 & \text{if } i \neq j. \end{cases}$$

Definition 6.4. Given a vector $\mathbf{v} = [v_1, v_2, \dots, v_n]^T \in \mathbb{F}_r^n$ and evaluation points $\{1, 2, \dots, n\}$, the *Lagrange interpolation* of $\mathbf{v}$, denoted $\mathcal{L}(\mathbf{v})(x)$, is the unique polynomial of degree at most $n - 1$ such that $\mathcal{L}(\mathbf{v})(i) = v_i$ for all $i \in \{1, 2, \dots, n\}$.

Remark 6.2. In practice, the evaluation points $\{1, 2, \dots, n\}$ are taken to be field elements in $\mathbb{F}_r$. For this to be well-defined, we require $n < r$ to ensure all evaluation points are distinct elements of $\mathbb{F}_r$.

Proposition 6.3 (Root bound). A nonzero polynomial of degree d in one variable over a field F has at most d roots in F , even counted with multiplicity.

Proof sketch. Dividing $p \in F[x]$ by $(x - a)$ gives $p(x) = q(x)(x - a) + r(x)$ with $r(x) = 0$ or $\deg r(x) < \deg(x - a) = 1$; in either case r is a constant. Evaluating at $x = a$ gives $r = p(a)$, so $p(a) = 0$ iff $(x - a) \mid p$. By induction roots $a_1, \dots, a_k$ (not necessarily distinct) give $(x - a_1) \cdots (x - a_k)$ as a factor of p . Since linear factors are irreducible and $F[x]$ is a unique factorization domain (§9.2 Corollary 4), at most d of them can occur, giving the bound.

From Abstract Algebra: §9.2 Theorem 3 (division step); §9.4 Proposition 9 (factor criterion); §9.5 Proposition 17 (root bound). $\square$

Remark 6.4. The field hypothesis is critical. Over $\mathbb{Z}_8$ (not a field as it contains zero divisors: $4 \cdot 2 = 8 \equiv 0$), $x^2 - 1$ has four roots $\{1, 3, 5, 7\}$ despite having degree 2. To see where the sketch breaks, factoring out the root $a_1 = 1$, we get $p(x) = x^2 - 1 = (x - 1)(x + 1)$, i.e., $q(x) = x + 1$. However, the inductive step requires every root $b \neq a_1$ of p to be a root of q : evaluating at b gives $0 = p(b) = (b - a_1)q(b)$ with $q(b) = b + 1$. Yet, for $b = 3$ we get $(3 - 1)(3 + 1) = 8 \equiv 0$ with neither factor zero. So 3 is a root of p but of neither linear factor: in a ring with zero divisors, a root of a product need not be a root of either factor, and the degree bound on the number of roots fails.

Theorem 6.5 (Existence and Uniqueness of Lagrange Interpolation). For any n distinct points (i, v_i) with $i \in \{1, 2, \dots, n\}$ and $v_i \in \mathbb{F}_r$, there exists a unique polynomial $p(x)$ of degree at most $n - 1$ such that $p(i) = v_i$ for all i .

Proof. Existence. The Lagrange basis polynomials are defined as:

$$\ell_i(x) = \prod_{\substack{j=1 \\ j \neq i}}^n \frac{x - j}{i - j} \quad (6.7)$$

for $i \in \{1, 2, \dots, n\}$. Each $\ell_i(x)$ has degree $n - 1$ and satisfies

$$\ell_i(j) = \delta_{ij} \quad \text{for all } j \in \{1, 2, \dots, n\},$$

where δ_{ij} is the Kronecker delta from Definition 6.3. The polynomial

$$p(x) = \sum_{i=1}^n v_i \ell_i(x) \quad (6.8)$$

satisfies $p(i) = v_i$ for all i , establishing existence.

Uniqueness. Suppose $p(x)$ and $q(x)$ are both polynomials of degree at most $n - 1$ such that $p(i) = q(i) = v_i$ for all i . Then $h(x) = p(x) - q(x)$ is a polynomial of degree at most $n - 1$ with n roots (at $x = 1, 2, \dots, n$). By Proposition 6.3, a non-zero polynomial of degree d has at most d roots. Since $h(x)$ has n roots but degree at most $n - 1$, we must have $h(x) = 0$, so $p(x) = q(x)$, establishing uniqueness. $\square$

Example 6.6. Consider the vector $\mathbf{v} = [4, 9, 16]^T$ with evaluation points $\{1, 2, 3\}$. We will reconstruct the polynomial $p(x)$ such that $p(1) = 4$, $p(2) = 9$, and $p(3) = 16$.

The Lagrange basis polynomials for $n = 3$ are:

$$\begin{aligned}\ell_1(x) &= \frac{(x-2)(x-3)}{(1-2)(1-3)} = \frac{(x-2)(x-3)}{2} = \frac{x^2 - 5x + 6}{2} \\ \ell_2(x) &= \frac{(x-1)(x-3)}{(2-1)(2-3)} = \frac{(x-1)(x-3)}{-1} = -(x^2 - 4x + 3) = -x^2 + 4x - 3 \\ \ell_3(x) &= \frac{(x-1)(x-2)}{(3-1)(3-2)} = \frac{(x-1)(x-2)}{2} = \frac{x^2 - 3x + 2}{2}\end{aligned}$$

The interpolated polynomial is:

$$\begin{aligned}\mathcal{L}(\mathbf{v})(x) &= 4 \cdot \ell_1(x) + 9 \cdot \ell_2(x) + 16 \cdot \ell_3(x) \\ &= 4 \cdot \frac{x^2 - 5x + 6}{2} + 9 \cdot (-x^2 + 4x - 3) + 16 \cdot \frac{x^2 - 3x + 2}{2} \\ &= 2x^2 - 10x + 12 - 9x^2 + 36x - 27 + 8x^2 - 24x + 16 \\ &= x^2 + 2x + 1\end{aligned}$$

We can verify: $\mathcal{L}(\mathbf{v})(1) = 1+2+1 = 4$, $\mathcal{L}(\mathbf{v})(2) = 4+4+1 = 9$, and $\mathcal{L}(\mathbf{v})(3) = 9+6+1 = 16$, confirming that the interpolation is correct.

Example 6.7 (Shamir's secret sharing). Lagrange interpolation gives *Shamir's k -of- n secret sharing*. To share a secret $s \in \mathbb{F}_r$, sample a random polynomial $p(x)$ of degree $k-1$ with $p(0) = s$ and distribute the shares $(i, p(i))$ for $i = 1, \dots, n$. Any k shares determine p uniquely (by the existence and uniqueness above), so the secret is recovered as $s = p(0) = \sum_i v_i \ell_i(0)$. Any $k-1$ shares reveal nothing: for each guessed secret $p(0)$ there is exactly one interpolating polynomial, so all guesses are equally consistent. Threshold schemes of this kind are used in distributed key generation and the multi-party ceremonies for trusted setups (Section 7).

The Lagrange interpolation polynomial is given by:

$$\mathcal{L}(\mathbf{v})(x) = \sum_{i=1}^n v_i \prod_{\substack{j=1 \\ j \neq i}}^n \frac{x-j}{i-j} \quad (6.9)$$

Proposition 6.8 (Homomorphic Properties of Lagrange Interpolation). For vectors $\mathbf{v}, \mathbf{w} \in \mathbb{F}_r^n$ and scalar $\lambda \in \mathbb{F}_r$:

1. $\mathcal{L}(\mathbf{v} + \mathbf{w})(x) = \mathcal{L}(\mathbf{v})(x) + \mathcal{L}(\mathbf{w})(x)$
2. $\mathcal{L}(\lambda \mathbf{v})(x) = \lambda \mathcal{L}(\mathbf{v})(x)$

Proof. The first property follows from the linearity of polynomial evaluation and the uniqueness of interpolation. For any evaluation point $i \in \{1, 2, \dots, n\}$:

$$\begin{aligned}\mathcal{L}(\mathbf{v} + \mathbf{w})(i) &= (v_i + w_i) \\ &= \mathcal{L}(\mathbf{v})(i) + \mathcal{L}(\mathbf{w})(i)\end{aligned}$$

Since both sides are polynomials of degree at most $n-1$ that agree on n points, they must be identical. The second property follows similarly. $\square$

These homomorphic properties enable converting vector operations into polynomial operations, which is fundamental for constructing Quadratic Arithmetic Programs, subsection 6.6.

6.4 The Schwartz-Zippel Lemma

Lemma 6.9 (Schwartz-Zippel). Let $p(x)$ and $q(x)$ be two distinct polynomials over $\mathbb{F}_r$ with degrees at most d_p and d_q respectively. If u is chosen uniformly at random from $\mathbb{F}_r$, then

$$\Pr[p(u) = q(u)] \leq \frac{\max(d_p, d_q)}{r} \quad (6.10)$$

This is the univariate case; the general Schwartz-Zippel lemma is multivariate, with bound $d/|S|$ for u drawn from a finite set S .

Proof. If $p(x) \neq q(x)$, then $h(x) = p(x) - q(x)$ is a non-zero polynomial of degree at most $d = \max(d_p, d_q)$. By Proposition 6.3 a non-zero polynomial of degree d has at most d roots in $\mathbb{F}_r$. Therefore, $\Pr[h(u) = 0] \leq d/r$, which implies $\Pr[p(u) = q(u)] \leq d/r$. $\square$

Corollary 6.10. For distinct polynomials $p(x) \neq q(x)$ of degree at most d over $\mathbb{F}_r$, $\Pr[p(u) = q(u)] \leq d/r$ for uniformly random $u \in \mathbb{F}_r$, which is negligible when $d \ll r$.

The Schwartz-Zippel Lemma enables efficient polynomial equality testing: instead of exchanging and comparing all $d + 1$ coefficients, each party evaluates at a common random point and exchanges the result, i.e., a single field element. Note, evaluation still costs $\mathcal{O}(d)$ field operations (via Horner's rule); only the final comparison is $\mathcal{O}(1)$. The saving is in communication (1 field element vs. $d + 1$), not computation; verification becomes constant-time once evaluations arrive precomputed inside commitments (Section 7). For example, if $r \approx 2^{254}$ and $d \leq 10^6$, then the error probability is at most $10^6/2^{254} \approx 2^{-234}$, which is cryptographically negligible (see Definition 1.8).

6.5 Vector Equality Testing via Polynomials

We can combine Lagrange interpolation with the Schwartz-Zippel Lemma to test vector equality efficiently. Given two vectors $\mathbf{v}, \mathbf{w} \in \mathbb{F}_r^n$, we can test if $\mathbf{v} = \mathbf{w}$ by:

1. Constructing polynomials $p(x) = \mathcal{L}(\mathbf{v})(x)$ and $q(x) = \mathcal{L}(\mathbf{w})(x)$
2. Selecting a random point $u \in \mathbb{F}_r$
3. Evaluating $p(u)$ and $q(u)$
4. Accepting $\mathbf{v} = \mathbf{w}$ if $p(r) = q(r)$, rejecting otherwise

By the Schwartz-Zippel Lemma (Lemma 6.9), if $\mathbf{v} \neq \mathbf{w}$, then $p(x) \neq q(x)$, and the probability of false acceptance is at most $(n - 1)/r$, which is negligible when $n \ll r$.

6.6 Quadratic Arithmetic Programs

Definition 6.5. Given an R1CS with matrices $\mathbf{L}, \mathbf{R}, \mathbf{O} \in \mathbb{F}_r^{m \times n}$, a *Quadratic Arithmetic Program* (QAP) is a tuple $Q = (\{L_j(x)\}_{j=1}^n, \{R_j(x)\}_{j=1}^n, \{O_j(x)\}_{j=1}^n, t(x))$ of polynomials over $\mathbb{F}_r[x]$ that encodes the R1CS constraint system. Given a witness vector $\mathbf{a} \in \mathbb{F}_r^n$, we obtain a triple of polynomials $(L(x), R(x), O(x)) \in \mathbb{F}_r[x]^3$; Q *accepts* $\mathbf{a}$ if $t(x)$ divides $L(x) \cdot R(x) - O(x)$.

The mapping is constructed as follows. First, define polynomials $L_j(x), R_j(x), O_j(x)$ for each column $j \in \{1, 2, \dots, n\}$ by applying Lagrange interpolation to the j -th column:

$$L_j(x) = \mathcal{L}([\mathbf{L}_{1,j}, \mathbf{L}_{2,j}, \dots, \mathbf{L}_{m,j}]^T)(x) \quad (6.11)$$

$$R_j(x) = \mathcal{L}([\mathbf{R}_{1,j}, \mathbf{R}_{2,j}, \dots, \mathbf{R}_{m,j}]^T)(x) \quad (6.12)$$

$$O_j(x) = \mathcal{L}([\mathbf{O}_{1,j}, \mathbf{O}_{2,j}, \dots, \mathbf{O}_{m,j}]^T)(x) \quad (6.13)$$

Then for witness vector $\mathbf{a} = [a_1, a_2, \dots, a_n]^T$ (with $a_1 = 1$), the QAP mapping is:

$$\text{QAP}(\mathbf{a}) = \left(\sum_{j=1}^n a_j L_j(x), \sum_{j=1}^n a_j R_j(x), \sum_{j=1}^n a_j O_j(x) \right) = (L(x), R(x), O(x)) \quad (6.14)$$

The linear combination $\sum_{j=1}^n a_j L_j(x)$ is well-defined by the homomorphic properties of Lagrange interpolation (Proposition 6.8), which ensure that vector addition and scalar multiplication are preserved under interpolation.

Note that $L(x)$, $R(x)$, and $O(x)$ each have degree at most $m - 1$ (since they interpolate m points). However, $L(x) \cdot R(x)$ has degree at most $2(m - 1) = 2m - 2$, while $O(x)$ has degree at most $m - 1$. For the equation $L(x) \cdot R(x) - O(x) = H(x) \cdot t(x)$ to hold, where $t(x)$ has degree m , the polynomial $H(x)$ must exist with degree at most $m - 2$. Whether such an $H(x)$ exists depends on the witness; Theorem 6.11 shows divisibility holds exactly when the R1CS is satisfied.

Theorem 6.11. An R1CS constraint $(\mathbf{L}\mathbf{a}) \circ (\mathbf{R}\mathbf{a}) = \mathbf{O}\mathbf{a}$ is satisfied if and only if there exists a polynomial $H(x)$ such that

$$L(x) \cdot R(x) - O(x) = H(x) \cdot t(x) \quad (6.15)$$

where $t(x) = \prod_{i=1}^m (x - i)$ is the *target polynomial*.

Proof. By construction, $L(x) = \sum_{j=1}^n a_j L_j(x)$ where $L_j(i) = \mathbf{L}_{i,j}$ (since $L_j(x)$ interpolates the j -th column of $\mathbf{L}$). By the homomorphic properties of Lagrange interpolation (Proposition 6.8), we have:

$$L(i) = \sum_{j=1}^n a_j L_j(i) = \sum_{j=1}^n a_j \mathbf{L}_{i,j} = \mathbf{L}_i \cdot \mathbf{a} \quad (6.16)$$

Similarly, $R(i) = \mathbf{R}_i \cdot \mathbf{a}$ and $O(i) = \mathbf{O}_i \cdot \mathbf{a}$ for each constraint $i \in \{1, 2, \dots, m\}$.

If the R1CS is satisfied, then $L(i) \cdot R(i) = O(i)$ for all i , which means the vector $[L(1) \cdot R(1) - O(1), L(2) \cdot R(2) - O(2), \dots, L(m) \cdot R(m) - O(m)]^T$ is the zero vector. This implies that $L(x) \cdot R(x) - O(x)$ has roots at $x = 1, 2, \dots, m$; the linear factors $(x - i)$ are pairwise coprime and each divides $L(x) \cdot R(x) - O(x)$, so their product $t(x) = \prod_{i=1}^m (x - i)$ divides it, and the quotient $H(x)$ exists. Since $L(x)$ and $R(x)$ have degree at most $m - 1$, their product $L(x) \cdot R(x)$ has degree at most $2m - 2$, while $O(x)$ has degree at most $m - 1$. Therefore, $L(x) \cdot R(x) - O(x)$ has degree at most $2m - 2$, and $H(x)$ has degree at most $(2m - 2) - (m) = m - 2$.

The polynomial $H(x)$ can be constructed by polynomial division: $H(x) = [L(x) \cdot R(x) - O(x)]/t(x)$. Alternatively, since $H(x)$ has degree at most $m - 2$, it can be determined by evaluating the quotient at $m - 1$ points distinct from the roots of $t(x)$ and applying Lagrange interpolation.

Conversely, if $L(x) \cdot R(x) - O(x) = H(x) \cdot t(x)$, then evaluating at $x = i$ gives $L(i) \cdot R(i) - O(i) = H(i) \cdot t(i) = 0$, so $L(i) \cdot R(i) = O(i)$ for all constraints i . $\square$

To understand why this construction works, consider the constraint vector $\mathbf{b} = [L(1) \cdot R(1) - O(1), L(2) \cdot R(2) - O(2), \dots, L(m) \cdot R(m) - O(m)]^T$. When the R1CS is satisfied, this vector is the zero vector: $\mathbf{b} = \mathbf{0}$, meaning $L(i) \cdot R(i) - O(i) = 0$ for all evaluation points $i \in \{1, 2, \dots, m\}$.

However, outside the evaluation set, the polynomial $L(x) \cdot R(x) - O(x)$ may be non-zero. Having roots at all m evaluation points acts as a kind of “destructive interference”: each root contributes a linear factor $(x - i)$, forcing $t(x) = \prod_{i=1}^m (x - i)$ to divide $L(x) \cdot R(x) - O(x)$ exactly. The quotient $H(x)$ then has degree at most $m - 2$, and the equality $L(x) \cdot R(x) - O(x) = H(x) \cdot t(x)$ holds everywhere, not just at the evaluation points.

This theorem enables efficient verification: instead of checking all m constraints separately, we verify the polynomial divisibility condition at a single random point using the Schwartz-Zippel Lemma (Lemma 6.9). Evaluating L , R , O , t at that point still costs $\mathcal{O}(m)$ operations,

so the gain at this stage is one check in place of m ; the verifier’s work becomes constant only when the evaluations are supplied precomputed in the CRS (Section 7).

6.7 Building Zero-Knowledge Proofs from R1CS

In this context, a *zero-knowledge* proof demonstrates that a system of constraints (polynomial equations) is satisfied by some witness, without revealing the witness. The prover convinces the verifier that such a witness exists while keeping it hidden.

To construct a zero-knowledge proof from an R1CS, we need to:

1. Convert the R1CS to QAP form, obtaining polynomials $L(x), R(x), O(x)$ and target polynomial $t(x)$ (the vanishing polynomial, often denoted $Z(x)$)
2. Have the prover demonstrate knowledge of a witness $\mathbf{a}$ such that $L(x) \cdot R(x) - O(x)$ is divisible by $t(x)$
3. Use cryptographic commitments (via elliptic curve points) to hide the witness while allowing verification
4. Replace the Hadamard product verification with bilinear pairings, which can verify a single multiplication in the exponent

Encoding the witness as $[a_1G, \dots, a_nG]^T$, the map $\varphi : k \mapsto kG$ (Theorem 1.4) preserves linear combinations, so $(\mathbf{L}_i \cdot \mathbf{a})G = \sum_{j=1}^n \mathbf{L}_{i,j}(a_jG)$ (and similarly for $\mathbf{R}\mathbf{a}, \mathbf{O}\mathbf{a}$). The constraint can be verified without revealing the witness via the pairing

$$e((\mathbf{L}_i \cdot \mathbf{a})G_1, (\mathbf{R}_i \cdot \mathbf{a})G_2) \stackrel{?}{=} e(G_1, G_2)^{(\mathbf{O}_i \cdot \mathbf{a})}. \quad (6.17)$$

This construction aims to achieve:

- *Completeness*: A valid witness always produces an accepting proof
- *Soundness*: An invalid witness is rejected with overwhelming probability (by Lemma 6.9)
- *Zero-Knowledge*: The proof reveals no information about the witness beyond its validity

As stated, however, soundness and zero-knowledge do not yet hold: a prover may submit arbitrary curve points satisfying the pairing check, and the deterministic encoding $[a_1G, \dots, a_nG]$ can be guess-and-checked when the witness has low entropy. Groth16 (Section 8) closes both gaps. Note also that the Schwartz–Zippel bound applies to a prover who commits to *polynomials*; a malicious prover instead chooses *group elements*, so the lemma alone is not a soundness proof. Soundness of the full protocol is Groth16’s Theorem 1, proven in the generic group model (see Attack 3, Section 8.1).

7 Trusted Setup and QAP Evaluation

The constructions in this section yield *zkSNARKs*: zero-knowledge proofs that are **succinct** (constant-size proofs, constant-time verification) and **non-interactive** (no back-and-forth between prover and verifier). The trade-off is a *trusted setup*: a one-time process that generates circuit-specific parameters. We do not evaluate the QAP polynomials in the clear (that would reveal the witness). The setup produces public parameters (the *common reference string*, CRS; contrasted with the SRS in Remark 7.2) that encode evaluation at a hidden point τ , so the prover can compute commitments and the verifier can check them without learning τ or the witness.

By contrast, *zk-STARKs* (**S**calable **T**ransparent **A**rguments of **K**nowledge) avoid circuit-specific toxic waste by relying on hash-based and oracle-style techniques; proofs are typically larger than a constant-size pairing SNARK and verifier work may scale with the instance, in exchange for transparency. This document develops pairing-based SNARKs (QAP evaluation and Groth16); STARKs are a complementary design point in the wider succinct-proof landscape. Groth16’s setup is additionally *circuit-specific*: the CRS depends on L, R, O , so changing the circuit requires a new ceremony; universal-setup schemes such as PLONK relax this at the cost of larger proofs.

7.1 Trusted Setup

Polynomial evaluation can be expressed as an inner product of coefficients with successive powers of the evaluation point: for $f(x) = c_d x^d + \dots + c_0$ and any $u \in \mathbb{F}_r$,

$$f(u) = \langle \mathbf{c}, [u^d, u^{d-1}, \dots, u, 1] \rangle. \quad (7.1)$$

Example 7.1. Let $f(x) = 3x^3 + 2x^2 + 5x + 10$. The coefficients are $\mathbf{c} = [3, 2, 5, 10]^T$. Then $f(2) = 3(2)^3 + 2(2)^2 + 5(2) + 10$ can also be written as

$$f(2) = \langle [3, 2, 5, 10], [8, 4, 2, 1] \rangle = 3 \cdot 8 + 2 \cdot 4 + 5 \cdot 2 + 10 \cdot 1. \quad (7.2)$$

A trusted party picks a secret $\tau \in \mathbb{F}_r$ (the *toxic waste*), computes $[\tau^d G_1, \dots, \tau G_1, G_1]$, and publishes it.

Definition 7.1 (Structured Reference String). Let $d \geq 1$ and let G_1 generate a cyclic group of prime order r . For secret $\tau \in \mathbb{F}_r$, the *structured reference string* (SRS) of degree d is

$$[\Omega_d, \Omega_{d-1}, \dots, \Omega_2, \Omega_1, G_1] = [\tau^d G_1, \tau^{d-1} G_1, \dots, \tau^2 G_1, \tau G_1, G_1], \quad (7.3)$$

with $\Omega_i = \tau^i G_1$. The secret τ is *encoded* in the SRS as group elements $\tau^i G_1$ rather than as raw scalars; the discrete-log problem ensures that neither the prover nor the verifier can recover τ from the published points. Given this public SRS, the prover can compute $f(\tau)G_1$ for any degree- $\leq d$ polynomial $f(x) = c_d x^d + \dots + c_0$ via

$$f(\tau)G_1 = \langle [c_d, \dots, c_1, c_0], [\Omega_d, \dots, \Omega_1, G_1] \rangle, \quad (7.4)$$

without knowing τ . Each power must be published separately: given only τG_1 , the prover cannot derive $\tau^2 G_1$ without knowing the scalar τ , so every $\tau^i G_1$ is required as its own SRS point.

Remark 7.2 (SRS vs. CRS). The Groth16 setup has two phases, with distinct outputs. The *powers of tau* phase produces the SRS above: group elements encoding powers of the secret τ , circuit-agnostic and reusable across all circuits up to the degree bound d . The *circuit-specific* phase takes the SRS and the circuit’s QAP as input and produces the *Common Reference String* (CRS): the proving and verification keys, comprising the SRS together with the auxiliary points and Ψ_j terms of Section 8. The CRS is circuit-specific but reusable across proofs for that circuit. Both phases require toxic-waste destruction. Where we write “SRS for $H(\tau)t(\tau)$ ” below, we mean a structured array of published points; since t is circuit-specific, such arrays belong to the CRS. (In the theoretical literature the terms are often interchangeable—an SRS is simply a CRS with hidden algebraic structure—but we keep the two-phase convention throughout.)

Example 7.3. Let $g(x) = 4x^2 + 7x + 8$, so the coefficient vector (with zero for the cubic term) is $[0, 4, 7, 8]$. Using the SRS $(\Omega_3, \Omega_2, \Omega_1, G_1)$ for degree 3, we have

$$g(\tau)G_1 = 0 \cdot \Omega_3 + 4 \cdot \Omega_2 + 7 \cdot \Omega_1 + 8 \cdot G_1 = 4\Omega_2 + 7\Omega_1 + 8G_1. \quad (7.5)$$

Thus $g(\tau)$ has been evaluated in the exponent without anyone (other than the setup creator) knowing τ .

The setup is *trusted* because the creator knows τ ; we assume they delete it. Multiparty generation below weakens this assumption.

Remark 7.4. The trusted setup is motivated by two concerns. First, the evaluation point τ must not be revealed: if the prover knew τ , they could fabricate polynomials that satisfy the QAP only at τ and break soundness; working in the exponent (publishing $\tau^i G_1$ instead of τ^i) keeps τ hidden. Second, the party generating the SRS might not do so honestly, so we need to verify that the published points have the correct structure (below) and may use a multiparty ceremony to reduce trust.

7.2 Toxic Waste and Forgery

This section references material outlined in this article by K. Gurban.

The setup process generates the *Common Reference String* (CRS), also called the proving and verification keys. These keys are circuit-specific: the prover uses the proving key to create proofs, and the verifier uses the verification key to check them. The random elements sampled during setup (τ , α , β , etc.) are called *toxic waste*: if a prover learns them, they can forge proofs and break soundness.

The secrecy of τ is critical as the verifier's check is performed at a *single* hidden point τ . The prover must supply $H(\tau)t(\tau)$ where $H = (L \cdot R - O)/t$; the numerator $L \cdot R - O$ is divisible by t only when the witness satisfies the constraints. Without knowing τ , a dishonest prover who submits an incorrect polynomial can only pass the check if the false polynomial happens to agree with the correct quotient at τ ; by the Schwartz–Zippel lemma, this occurs with probability at most d/r (where d is the polynomial degree), which is negligible when r is cryptographically large. If τ is exposed, however, a malicious prover can forge a proof: since τ is not a root of t (τ is uniform, so $t(\tau) = 0$ with probability m/r , negligible), the forger chooses arbitrary $L(\tau)$, $R(\tau)$, $O(\tau)$ (e.g., from fake inputs), then sets $H(z)$ to the *constant* polynomial $H(z) = (L(\tau)R(\tau) - O(\tau))/t(\tau)$. This H satisfies the verifier's check at τ but does not equal $(L \cdot R - O)/t$ as a polynomial—the prover never needed a valid witness.

For an honest prover with a valid witness, $H(z) = (L(z)R(z) - O(z))/t(z)$ is a polynomial in z , typically *non-constant* for a non-trivial circuit ($\deg(L) + \deg(R) - \deg(t) > 0$). The forger's $H(z)$, by contrast, is *constant*: it equals the scalar $(L(\tau)R(\tau) - O(\tau))/t(\tau)$, which does not depend on z . The verifier only checks at τ , so they cannot distinguish the constant from the true quotient.

Knowledge of τ is what enables the forgery. Without τ , the prover only sees group elements $[\tau^i]_1$, $[t(\tau)]_1$, etc.; they cannot freely choose arbitrary scalars $L(\tau)$, $R(\tau)$, $O(\tau)$ —the CRS structure constrains what they can produce. With τ , the prover can work in the exponent (with scalars) instead of being constrained by the CRS: they compute $t(\tau)$, pick fake values, and construct the proof elements that pass verification.

When τ is known, the forger need not construct polynomials $L(z)$, $R(z)$, $O(z)$ that satisfy the circuit. They only need scalar values $L(\tau)$, $R(\tau)$, $O(\tau)$ at the single point τ ; they can choose these arbitrarily (e.g. 1, 2, 5) and then set H so the check passes. The CRS gives them the structure to encode any such scalars into the proof.

Example 7.5. For a multiplication constraint $a \cdot b = c$, suppose $t(z) = z - 1$. A forger who knows τ picks $L(\tau) = 1$, $R(\tau) = 2$, $O(\tau) = 5$ (invalid: $1 \cdot 2 \neq 5$), sets $H(z) = (2 - 5)/t(\tau) = -3/t(\tau)$ (constant), and constructs the proof. The verifier's check $L(\tau)R(\tau) = O(\tau) + H(\tau)t(\tau)$ holds at τ , but the prover never had a valid witness.

Remark 7.6 (Multiparty ceremonies). For public circuits, trust in the setup can be distributed through a *multi-party computation* (MPC) ceremony: multiple participants each contribute randomness and destroy their share. It suffices that *one* honest participant discards their randomness for the combined toxic waste to remain unknown. Notable examples include ZCash's

“Powers of Tau” ceremony and “The Ceremony” (ZCash’s first MPC setup). ZCash’s first version (Sprout) used a Pinocchio-style construction (BCTV14); with Sapling (2018) it moved to Groth16, which improves upon these while retaining the need for a trusted setup.

Remark 7.7 (Verifying the SRS). The published SRS can itself be verified: if the setup also publishes $\Theta = \tau G_2$, a verifier can check the structure without learning τ : for $i \in \{1, \dots, d-1\}$, $e(\Omega_i, \Theta) \stackrel{?}{=} e(\Omega_{i+1}, G_2)$ (both sides equal $e(G_1, G_2)^{\tau^{i+1}}$ by bilinearity), and $e(G_1, \Theta) \stackrel{?}{=} e(\Omega_1, G_2)$.

Proposition 7.8 (Powers of tau ceremony). Multiple parties can generate the SRS so that if at least one is honest and discards their secret, the toxic waste is unknown. Alice samples her secret τ_1 and publishes $[\Omega_d, \dots, \Omega_1, G_1]$ with $\Omega_i = \tau_1^i G_1$ and $\Theta = \tau_1 G_2$; Bob verifies with the pairing checks, then samples his secret τ_2 and publishes $[\tau_2^d \Omega_d, \dots, \tau_2 \Omega_1, G_1]$ and $\tau_2 \Theta$. The new SRS has discrete logs $([(\tau_1 \tau_2)^d, \dots, (\tau_1 \tau_2), 1], \tau_1 \tau_2)$, i.e., the effective trapdoor is $\tau = \tau_1 \tau_2$; G_1 is unchanged. If either discards their secret, τ is unrecoverable. The process extends to more participants, participant i contributing τ_i with final trapdoor $\tau = \prod_i \tau_i$ (*powers of tau ceremony*).

7.3 Evaluating a Trusted Setup on a QAP

The prover shows that $L(\tau)R(\tau) = O(\tau) + H(\tau)t(\tau)$ (Theorem 6.11) at the secret τ without revealing the witness; the proof has constant size. This yields a *Pinocchio-style* proof (from “Pinocchio: Nearly Practical Verifiable Computation”), which ZCash used in its first version. Recall: m constraints (rows), n columns (witness elements). All degree bounds depend on m , not n : each column is interpolated through m evaluation points, giving basis polynomials of degree $\leq m-1$; L, R, O (as linear combinations of these) also have degree $\leq m-1$; the target t (or vanishing) polynomial has degree m (m roots, one per constraint point); and $H = (L \cdot R - O)/t$ has degree $\leq 2(m-1) - m = m-2$. The number of columns n determines how many basis polynomials exist, not their degree.

For L, R , and O , the setup publishes $[\Omega_{m-1}, \dots, \Omega_1, G_1]$ and $[\Theta_{m-1}, \dots, \Theta_1, G_2]$ (Definition 7.1). From the witness and QAP basis, the prover has coefficients l_k, r_k, o_k for L, R, O and computes

$$[A]_1 = L(\tau)G_1 = \langle [l_{m-1}, \dots, l_0], [\Omega_{m-1}, \dots, G_1] \rangle, \quad (7.6)$$

$$[B]_2 = R(\tau)G_2 = \langle [r_{m-1}, \dots, r_0], [\Theta_{m-1}, \dots, G_2] \rangle, \quad (7.7)$$

$$[O]_1 = O(\tau)G_1 = \langle [o_{m-1}, \dots, o_0], [\Omega_{m-1}, \dots, G_1] \rangle. \quad (7.8)$$

Subscripts 1 and 2 denote the group.

For the $H(\tau)t(\tau)$ term, the verifier needs a single G_1 element; we cannot send $H(\tau)$ and $t(\tau)$ separately and multiply (or pair) them, as that would not yield the required G_1 element, i.e., respectively, G_1 is an additive group and a pairing would land in the target group G_T , not G_1 . Since $H(x) \cdot t(\tau) = \sum_{k=0}^{m-2} h_k t(\tau) x^k$, evaluating at $x = \tau$ gives $H(\tau)t(\tau) = \langle [h_{m-2}, \dots, h_0], [\tau^{m-2} t(\tau), \dots, t(\tau)] \rangle$. The setup knows $t(x)$ and computes $t(\tau)$ before discarding τ ; it publishes $[\Upsilon_{m-2}, \dots, \Upsilon_0] = [\tau^{m-2} t(\tau) G_1, \dots, t(\tau) G_1]$. The prover then has $[H(\tau)t(\tau)]_1 = \langle [h_{m-2}, \dots, h_0], [\Upsilon_{m-2}, \dots, \Upsilon_0] \rangle$ and sets $[C]_1 = [O]_1 + [H(\tau)t(\tau)]_1$.

Finally, in this Pinocchio-style scheme, the verifier checks

$$e([A]_1, [B]_2) \stackrel{?}{=} e([C]_1, G_2). \quad (7.9)$$

By bilinearity,

$$e([A]_1, [B]_2) = e(L(\tau)G_1, R(\tau)G_2) = e(G_1, G_2)^{L(\tau) \cdot R(\tau)}, \quad (7.10)$$

and

$$e([C]_1, G_2) = e((O(\tau) + H(\tau)t(\tau))G_1, G_2) = e(G_1, G_2)^{O(\tau) + H(\tau)t(\tau)} \quad (7.11)$$

where the check therefore verifies $L(\tau)R(\tau) = O(\tau) + H(\tau)t(\tau)$ in the exponent. The proof is $([A]_1, [B]_2, [C]_1)$ —three group elements, independent of R1CS size. Only the prover’s work grows with the circuit; the verifier’s cost is constant.

Remark 7.9. Evaluating the QAP at a *hidden* point τ (via the SRS) is essential: if τ were public, a prover could construct polynomials that pass the check at τ without a valid witness. This Pinocchio-style scheme is still not a full ZK-SNARK—a prover could send arbitrary curve points that satisfy the pairing equation without knowing a witness. Groth16 adds further structure for soundness and zero-knowledge (Section 8).

Remark 7.10 (Why scalar field is $\mathbb{F}_r$, not $\mathbb{F}_p$). The groups G_1 and G_2 have order r , so $[k]G_1 = [k \bmod r]G_1$. Every scalar that multiplies a group element—witness coefficients, QAP evaluations, setup secrets—is therefore only meaningful modulo r , and the natural field for all proof-system arithmetic is $\mathbb{F}_r$. If one used $\mathbb{F}_p$ instead, polynomial arithmetic would reduce modulo p while the group reduces modulo r ; since $p \neq r$ in general, a QAP identity that holds over $\mathbb{F}_p[x]$ need not hold when checked via the pairing, and vice versa, breaking soundness.

8 Groth16

Groth16 is a canonical *SNARK* in the sense of Section 7: a succinct non-interactive argument whose security here rests on pairings and a structured reference string. In particular, the Groth16 proof itself has constant size: the prover sends exactly three group elements $([A]_1, [B]_2, [C']_1)$ regardless of circuit size. The Pinocchio-style scheme in Section 7.3 is forgeable (Section 8.1); Groth16 adds auxiliary points from the trusted setup to prevent this; the following subsections are built on parts of the following article RareSkills (2023), and, naturally, Jens Groth’s paper On the Size of Pairing-based Non-interactive Arguments, published Eurocrypt 2016.

Remark 8.1 (Improvements). Considerable improvements have been made in the decade since the original protocol was published, most recently, as of the time of writing, BABE: Verifying Proofs on Bitcoin Made 1000x Cheaper, which aims to tailor a verification-size optimization for a BitVM solution.

8.1 Introducing α and β

The verification equation (7.9) is forgeable: a prover can pick arbitrary a, b, c with $ab = c$ and present $([a]_1, [b]_2, [c]_1)$, satisfying the pairing check without a valid witness. To prevent this, we add an auxiliary term $[D]_{12} = e([A]_1, [B]_2)$ with unknown discrete logarithm, so the verifier instead checks $e([A]_1, [B]_2) \stackrel{?}{=} [D]_{12} \cdot e([C]_1, G_2)$. In the expanded scheme, α and β act as secret *scalar shifts*: we replace $L(\tau)$ and $R(\tau)$ by their translates $\alpha + L(\tau)$ and $\beta + R(\tau)$ respectively. Since α and β are unknown to the prover, the shift cannot be undone: the prover cannot factor the product $(\alpha + L(\tau))(\beta + R(\tau))$ into parts they control, nor deduce the shifted values from the published points. We consider three attack scenarios. The first two fail, illustrating what α and β prevent; the third succeeds and motivates the γ/δ separation of Section 8.4.

Attack 1 (*forge A, B , derive C*). A malicious prover picks a', b' and computes $[A]_1, [B]_2$. To satisfy the equation they would need a point $[C']_1$ such that

$$e([C']_1, G_2) = e([A]_1, [B]_2) \cdot [D]_{12}^{-1}.$$

The right-hand side is an element of the target group; extracting a corresponding G_1 point from it would amount to solving a discrete-log-type problem, which is infeasible.

Attack 2 (forge C , derive A, B). The prover picks c' and computes $[C']_1$. They need $[A]_1, [B]_2$ such that $e([A]_1, [B]_2) = \zeta_{12}$ where $\zeta_{12} = [D]_{12} \cdot e([C']_1, G_2)$ is known. No efficient algorithm is known that, given $\zeta_{12} \in G_T$, finds $[A]_1, [B]_2$ such that $e([A]_1, [B]_2) = \zeta_{12}$, i.e., decomposes ζ_{12} back into a pairing of G_1 and G_2 elements (the *pairing inversion* problem).

Attack 3 (reuse CRS elements). Instead of choosing fresh scalars, the forger builds the proof from published points: for any scalar b , the triple $\pi = ([\alpha]_1, [\beta]_2 + bG_2, b[\alpha]_1)$ verifies, since

$$e(\alpha G_1, (\beta + b)G_2) = e(G_1, G_2)^{\alpha\beta + \alpha b} = e([\alpha]_1, [\beta]_2) \cdot e(b[\alpha]_1, G_2),$$

with every component computable from the CRS and no witness used. The scheme as stated has no defense against this attack. It is closed by the γ/δ separation (Section 8.4), and ultimately by Groth16's soundness proof in the generic group model¹⁷.

In practice the trusted setup samples random $\alpha, \beta \in \mathbb{F}_r$ and publishes $[\alpha]_1 = \alpha G_1$ and $[\beta]_2 = \beta G_2$; $[D]_{12} = e([\alpha]_1, [\beta]_2)$ is never computed explicitly.

8.2 Expanded QAP, Trusted Setup, and Prover

To make the verification $e([A]_1, [B]_2) \stackrel{?}{=} e([\alpha]_1, [\beta]_2) \cdot e([C]_1, G_2)$ realisable by the prover, we modify the QAP to incorporate α and β by *translating* L and R . From Theorem 6.11, $L(x)R(x) = O(x) + H(x)t(x)$. Introduce auxiliary scalars θ, η and apply translation to the left-hand side:

$$\begin{aligned} (\theta + L(x))(\eta + R(x)) &= \theta\eta + \theta R(x) + \eta L(x) + L(x)R(x) \\ &= \theta\eta + \theta R(x) + \eta L(x) + O(x) + H(x)t(x). \end{aligned} \quad (8.1)$$

Replacing $\theta \mapsto \alpha$ and $\eta \mapsto \beta$, and evaluating at the hidden point τ , we obtain the expanded QAP identity

$$(\alpha + L(\tau))(\beta + R(\tau)) = \alpha\beta + \alpha R(\tau) + \beta L(\tau) + O(\tau) + H(\tau)t(\tau). \quad (8.2)$$

Define the translated elements

$$[A]_1 := [\alpha]_1 + L(\tau)G_1 = (\alpha + L(\tau))G_1, \quad [B]_2 := [\beta]_2 + R(\tau)G_2 = (\beta + R(\tau))G_2, \quad (8.3)$$

i.e. $[A]_1$ and $[B]_2$ are the group elements corresponding to the translates $\alpha + L(\tau)$ and $\beta + R(\tau)$. By bilinearity of the pairing $e : G_1 \times G_2 \rightarrow G_T$, we have $e(aG_1, bG_2) = e(G_1, G_2)^{ab}$ for scalars a, b . Thus

$$e([A]_1, [B]_2) = e((\alpha + L(\tau))G_1, (\beta + R(\tau))G_2) = e(G_1, G_2)^{(\alpha + L(\tau))(\beta + R(\tau))}. \quad (8.4)$$

Expanding the exponent and applying (8.2), we obtain

$$e([A]_1, [B]_2) = e(G_1, G_2)^{\alpha\beta + \alpha R(\tau) + \beta L(\tau) + O(\tau) + H(\tau)t(\tau)} = e([\alpha]_1, [\beta]_2) \cdot e([C]_1, G_2), \quad (8.5)$$

where $[C]_1 := (\alpha R(\tau) + \beta L(\tau) + O(\tau) + H(\tau)t(\tau))G_1$. The verifier therefore checks

$$e([A]_1, [B]_2) \stackrel{?}{=} e([\alpha]_1, [\beta]_2) \cdot e([C]_1, G_2). \quad (8.6)$$

The prover can compute $[A]_1$ and $[B]_2$ from the SRS (Definition 7.1) without knowing τ , α , or β : $[A]_1 = [\alpha]_1 + \langle l_{m-1}, \dots, l_0, [\Omega_{m-1}, \dots, G_1] \rangle$ and similarly for $[B]_2$ using the G_2 SRS.

¹⁷J. Groth, *On the Size of Pairing-Based Non-interactive Arguments*, EUROCRYPT 2016, <https://eprint.iacr.org/2016/260>, Theorem 1.

However, $[C]_1$ requires $\alpha R(\tau) + \beta L(\tau) + O(\tau)$; the prover cannot pair $[\alpha]_1$ with $R(\tau)$ (or $[\beta]_2$ with $L(\tau)$) because that would yield an element of G_T , not G_1 . The trusted setup therefore precomputes, for each column $j \in \{1, \dots, n\}$,

$$\Psi_j(x) := \alpha R_j(x) + \beta L_j(x) + O_j(x), \quad (8.7)$$

evaluates $\Psi_j(\tau)$ at setup time, and publishes $[\Psi_j(\tau)]_1$. Since $\alpha R(\tau) + \beta L(\tau) + O(\tau) = \sum_{j=1}^n a_j \Psi_j(\tau)$, the prover computes $[C]_1 = \sum_{j=1}^n a_j [\Psi_j(\tau)]_1 + [H(\tau)t(\tau)]_1$ using the witness $\mathbf{a}$ and the $H(\tau)t(\tau)$ SRS (as in Section 7.3).

8.3 Public Inputs, Zero-Knowledge, and Verification

The verification above does not support *public inputs*, i.e. making a portion of the witness public. Some public inputs are necessary: they commit the prover to a specific claim (e.g., an output value, a vote choice, or a Merkle root). Without them, the verifier would not know what statement is being proven—the proof could attest to any satisfying witness. By convention, the first ℓ elements of $\mathbf{a}$ are public. The prover reveals $(a_1, \dots, a_\ell)$; the verifier must incorporate these into the check.

Only the computation of the G_1 contribution changes. Denote by $[C']_1$ the *private* contribution (a distinct point from the full $[C]_1$): the prover uses only indices $j \in \{\ell + 1, \dots, n\}$,

$$[C']_1 := \sum_{j=\ell+1}^n a_j [\Psi_j(\tau)]_1 + [H(\tau)t(\tau)]_1, \quad (8.8)$$

while $[A]_1$ and $[B]_2$ still use the full witness. The verifier computes the *public* contribution

$$[X]_1 := \sum_{j=1}^{\ell} a_j [\Psi_j(\tau)]_1 \quad (8.9)$$

from the revealed $(a_1, \dots, a_\ell)$ and the published $[\Psi_j(\tau)]_1$ for $j \leq \ell$.

Proposition 8.2 (Decomposition of the full G_1 contribution). Let $[C]_1 := \sum_{j=1}^n a_j [\Psi_j(\tau)]_1 + [H(\tau)t(\tau)]_1$ denote the full point from the scheme without public inputs. Then $[C]_1 = [X]_1 + [C']_1$ in G_1 .

Proof. Let $\varphi(k) := kG_1$ (Theorem 1.4). Define scalars $x := \sum_{j=1}^{\ell} a_j \Psi_j(\tau)$, $c' := \sum_{j=\ell+1}^n a_j \Psi_j(\tau) + H(\tau)t(\tau)$, and $c := \sum_{j=1}^n a_j \Psi_j(\tau) + H(\tau)t(\tau)$. Then $[X]_1 = \varphi(x)$, $[C']_1 = \varphi(c')$, and $[C]_1 = \varphi(c)$. Since $x + c' = c$ (splitting the sum over j), the homomorphism gives

$$[X]_1 + [C']_1 = \varphi(x) + \varphi(c') = \varphi(x + c') = \varphi(c) = [C]_1. \quad (8.10)$$

□

By Proposition 8.2 and bilinearity, $e([X]_1 + [C']_1, G_2) = e([X]_1, G_2) \cdot e([C']_1, G_2)$. The verification equation becomes

$$e([A]_1, [B]_2) \stackrel{?}{=} e([\alpha]_1, [\beta]_2) \cdot e([X]_1, G_2) \cdot e([C']_1, G_2). \quad (8.11)$$

8.4 Separating Public and Private Inputs: γ and δ

The public part of the witness commits the prover to a specific claim: for example, the output of a computation, a vote choice, or a Merkle root. Without a binding between the proof and these public values, a prover could supply a valid proof while misrepresenting the claim—the verifier would accept the pairing equation but could not confirm that the proof corresponds to

the intended statement. For instance, a prover who knows x with $\text{hash}(x) = H$ could produce a proof and then claim the public output is some $H' \neq H$, instead of H , shifting H into the private sum; the verification equation would still hold, but the proof would no longer attest to the correct claim.

The scheme above assumes the prover uses only Ψ_j for $j \in \{\ell + 1, \dots, n\}$ when computing $[C']_1$. A dishonest prover can instead use $\Psi_1, \dots, \Psi_\ell$ in $[C']_1$, shifting public values into the private sum and forging a proof for an invalid witness.

Example 8.3 (Forgery by shifting public to private). Let $\mathbf{a} = [1, 2, 3, 4, 5]^T$ and $\ell = 3$, so the public part is $(1, 2, 3)$ and the private part is $(4, 5)$. The honest equation is

$$e([A]_1, [B]_2) = e([\alpha]_1, [\beta]_2) \cdot e((1[\Psi_1]_1 + 2[\Psi_2]_1 + 3[\Psi_3]_1), G_2) \cdot e((4[\Psi_4]_1 + 5[\Psi_5]_1 + [H(\tau)t(\tau)]_1), G_2).$$

A dishonest prover can claim public inputs $(1, 2, 0)$ and use $3[\Psi_3]_1$ inside $[C']_1$:

$$e([A]_1, [B]_2) = e([\alpha]_1, [\beta]_2) \cdot e((1[\Psi_1]_1 + 2[\Psi_2]_1 + 0[\Psi_3]_1), G_2) \cdot e((3[\Psi_3]_1 + 4[\Psi_4]_1 + 5[\Psi_5]_1 + [H(\tau)t(\tau)]_1), G_2).$$

The equation still holds, but the witness does not satisfy the original constraints.

To prevent this, the trusted setup introduces scalars $\gamma \neq 0$ and $\delta \neq 0$ and divides the public and private contributions by different factors. The split index ℓ (number of public witness elements) is fixed at circuit design time and known to the setup. The public Ψ_j (for $j \leq \ell$) are scaled by γ^{-1} , and the private Ψ_j (for $j > \ell$) and the $H(\tau)t(\tau)$ term by δ^{-1} . If γ and δ have unknown discrete logarithms, the prover cannot mix public and private terms.

The trusted setup samples $\alpha, \beta, \tau, \gamma, \delta$ uniformly at random from $\mathbb{F}_r$; the scalars are discarded (toxic waste). The following are *all* published:

- SRS for G_1 and G_2 (powers of τ)
- SRS for $H(\tau)t(\tau)$ divided by δ : $[\frac{\tau^{m-2}t(\tau)}{\delta}, \dots, \frac{t(\tau)}{\delta}]G_1$
- $[\Psi_j(\tau)]_1 = \frac{\Psi_j(\tau)}{\sigma_j}G_1$ where $\sigma_j = \begin{cases} \gamma & \text{if } j \leq \ell \text{ (public),} \\ \delta & \text{if } j > \ell \text{ (private).} \end{cases}$
- The auxiliary points $[\alpha]_1, [\beta]_1, [\beta]_2, [\gamma]_2, [\delta]_1, [\delta]_2$ (the scalars remain secret; $[\delta]_1$ and $[\beta]_1$ are needed for zero-knowledge in Section 8.5)

The prover's computation is unchanged: $[A]_1 = [\alpha]_1 + L(\tau)G_1$, $[B]_2 = [\beta]_2 + R(\tau)G_2$, and $[C']_1 = \sum_{j=\ell+1}^n a_j[\Psi_j(\tau)]_1 + [H(\tau)t(\tau)/\delta]_1$.

The verifier computes $[X]_1 = \sum_{j=1}^\ell a_j[\Psi_j(\tau)]_1$ from the public inputs and checks

$$e([A]_1, [B]_2) \stackrel{?}{=} e([\alpha]_1, [\beta]_2) \cdot e([X]_1, [\gamma]_2) \cdot e([C']_1, [\delta]_2). \quad (8.12)$$

The scalars γ and δ are discarded at setup, but the group elements $[\gamma]_2$ and $[\delta]_2$ are published in the CRS; the verifier uses these points in the pairing. The pairings with $[\gamma]_2$ and $[\delta]_2$ cancel the denominators in the published Ψ_j and $H(\tau)t(\tau)$ terms. In principle γ may be normalized to 1 (so $[\gamma]_2 = G_2$); Groth's paper notes either γ or δ can be normalized this way.

8.5 Zero-Knowledge: r and s

The scheme above is not yet zero-knowledge when the witness has private elements with few valid values (e.g. a vote choice from $\{A, B, C\}$). An attacker can exhaustively try each possible private value, run the prover to obtain a proof for each, and compare to the observed proof. Without salt, the proof is deterministic for a given witness, so the matching proof reveals the private value. The translation by α and β (Section 8.1) prevents forgery but does not prevent this linking attack: α and β are fixed at setup time and encoded in the CRS, so the attacker can reproduce any proof for any witness using the published CRS; the proof is uniquely determined by the witness.

Example 8.4 (Exhaustive search attack). Suppose the claim is that $x_1, x_2 \in \{0, 1\}$, the circuit is:

$$x_1(x_1 - 1) + x_2(x_2 - 1) = 0$$

An attacker need only try four combinations, generate a proof for each, and see which matches the prover's proof. The match reveals (x_1, x_2) .

Definition 8.1 (Salt). In a zero-knowledge proof, a *salt* (or blinding) factor is a random value sampled by the prover and incorporated into the proof so that multiple proofs for the same witness are statistically indistinguishable; an attacker cannot link a proof to a specific witness.

Salt is sampled *per proof* by the prover, unlike α and β which are fixed at setup. With random r, s per proof, the attacker cannot reproduce the exact proof for a guessed witness—they would need to know the prover's r, s , which they do not. The comparison attack therefore fails. Further, for a fixed witness, r makes $[A]_1$ uniformly distributed in G_1 and s makes $[B]_2$ uniform in G_2 , and $[C']_1$ is then the unique element making verification pass. The proof distribution is therefore identical for every valid witness (perfect zero-knowledge), and could be produced by a simulator knowing only the statement. With respect to Example 8.4 above, the prover attests that they have a valid choice without revealing what it was—which is zero-knowledge.

To prevent this, the prover *salts* the proof (Definition 8.1) with random $r, s \in \mathbb{F}_r$.¹⁸ Define

$$[A]_1 := [\alpha]_1 + L(\tau)G_1 + r[\delta]_1, \quad (8.13)$$

$$[B]_2 := [\beta]_2 + R(\tau)G_2 + s[\delta]_2, \quad (8.14)$$

$$[B]_1 := [\beta]_1 + R(\tau)G_1 + s[\delta]_1 = BG_1, \quad (8.15)$$

where $B = \beta + R(\tau) + s\delta$ is the discrete log of $[B]_2 = BG_2$ (needed for $[C']_1$). The trusted setup (Section 8.4) already publishes $[\delta]_1$ and $[\beta]_1$, which the prover needs to compute $[A]_1$ and $[B]_1$. Expanding $(\alpha + L(\tau) + r\delta)(\beta + R(\tau) + s\delta)$:

$$(\alpha + L + r\delta)(\beta + R + s\delta) = \alpha\beta + \alpha R + \beta L + LR + \alpha s\delta + Ls\delta + r\delta\beta + r\delta R + rs\delta^2.$$

Using $LR = O + Ht$, the first four terms become $\alpha\beta + \alpha R + \beta L + O + Ht$ (the original QAP identity). The remaining terms $(\alpha + L)s\delta + (\beta + R)r\delta + rs\delta^2$ can be rewritten as $As\delta + Br\delta - rs\delta^2$ (since $A = \alpha + L + r\delta$ and $B = \beta + R + s\delta$). The sum $\alpha R + \beta L + O$ equals $\sum_{j=1}^n a_j \Psi_j(\tau)$; we split at index ℓ and divide by γ (public) and δ (private) as in Section 8.4. The salt terms $As\delta + Br\delta - rs\delta^2$ belong to the private part. So the final scalar equation for $(\alpha + L + r\delta)(\beta + R + s\delta) =$

$$\underbrace{\alpha\beta + \gamma \cdot \frac{\sum_{j=1}^{\ell} a_j \Psi_j(\tau)}{\gamma}}_{\text{public}} + \underbrace{\delta \cdot \frac{\sum_{j=\ell+1}^n a_j \Psi_j(\tau) + H(\tau)t(\tau) + As\delta + Br\delta - rs\delta^2}{\delta}}_{\text{private}}. \quad (8.16)$$

The private scalar (right-hand underbrace) equals $\frac{\sum_{j=\ell+1}^n a_j \Psi_j(\tau) + H(\tau)t(\tau)}{\delta} + As + Br - rs\delta$ after dividing through by δ . In G_1 : the first term is the existing private sum; As becomes $s[A]_1$, Br becomes $r[B]_1$, and $rs\delta$ becomes $rs[\delta]_1$. Thus the prover sets

$$[C']_1 := \sum_{j=\ell+1}^n a_j [\Psi_j(\tau)]_1 + [H(\tau)t(\tau)/\delta]_1 + s[A]_1 + r[B]_1 - rs[\delta]_1. \quad (8.17)$$

The *prover* provides $[A]_1, [B]_2, [C']_1$. The *verifier* computes $[X]_1 = \sum_{j=1}^{\ell} a_j [\Psi_j(\tau)]_1$ from the public inputs and checks

$$e([A]_1, [B]_2) \stackrel{?}{=} e([\alpha]_1, [\beta]_2) \cdot e([X]_1, [\gamma]_2) \cdot e([C']_1, [\delta]_2). \quad (8.18)$$

¹⁸The salt scalars r, s are *not* the subgroup order r that indexes $\mathbb{F}_r$; the reuse of the letter is standard Groth16 notation.

The verification equation is unchanged; the extra terms $As\delta + Br\delta - rs\delta^2$ are absorbed into the private part and cancelled by pairing with $[\delta]_2$. With random r, s , each valid witness yields many possible proofs, so an attacker cannot link a proof to a specific witness.

Remark 8.5 (CRS split into proving and verification keys). In practice, the setup output (the CRS, Remark 7.2) is split by role into a *proving key* (PK) and a *verification key* (VK). The prover uses the large PK (including private-index Ψ_j terms for $j > \ell$ and $H(\tau)t(\tau)/\delta$ material), while the verifier only needs a much smaller subset: $[\alpha]_1, [\beta]_2, [\gamma]_2, [\delta]_2$, and the public-index elements $[\Psi_j(\tau)]_1$ for $j \leq \ell$. Therefore the verifier can accept a compact verification key rather than the full CRS.

8.6 Several Known Security Issues

Definition 8.2 (Malleable proof system). A proof system is *malleable* if, given a valid proof π for a statement x , an attacker can efficiently produce a different valid proof π' for x without knowing the witness.

Proposition 8.6 (Groth16 is malleable). Groth16 is malleable (Definition 8.2).

Proof. Given a valid proof $\pi = ([A]_1, [B]_2, [C']_1)$ for a statement with public inputs $(a_1, \dots, a_\ell)$, define $\pi' = (-[A]_1, -[B]_2, [C']_1)$. The verification equation checks $e([A]_1, [B]_2) \stackrel{?}{=} e([\alpha]_1, [\beta]_2) \cdot e([X]_1, [\gamma]_2) \cdot e([C']_1, [\delta]_2)$. For π' , the left-hand side is $e(-[A]_1, -[B]_2)$. By bilinearity, for scalars a, b we have $e(-aG_1, -bG_2) = e(G_1, G_2)^{(-a)(-b)} = e(G_1, G_2)^{ab} = e(aG_1, bG_2)$, so $e(-[A]_1, -[B]_2) = e([A]_1, [B]_2)$. The right-hand side is unchanged since $[C']_1$ and $[X]_1$ are the same. Thus π' verifies. The transformation requires only point negation, which is efficient. The attacker needs no knowledge of the witness. More generally, $(c[A]_1, c^{-1}[B]_2, [C']_1)$ verifies for any $c \neq 0$, since $e(cA, c^{-1}B) = e(G_1, G_2)^{c \cdot c^{-1} \cdot ab} = e(A, B)$; the negation attack is the case $c = -1$. $\square$

A minimal verification that the pairing is unchanged under negation:

```
Input: A in G1, B in G2 (arbitrary)
A' := neg(A); B' := neg(B)
Assert: pairing(A, B) = pairing(A', B')
```

Example 8.7 (Malleability attack on proof uniqueness). Suppose a system rewards one unit per valid proof of “I own ticket X ”. Alice creates a proof π and submits it to claim her reward. Bob observes π (proofs are public). Bob computes $\pi' = (-[A]_1, -[B]_2, [C']_1)$, which also verifies. Bob submits π' and claims a reward. Bob never knew the witness—he only transformed Alice’s proof. The system cannot distinguish π from π' ; if it accepts both, Bob double-claims. Applications that require proof uniqueness must bind proofs to a nonce or use additional mechanisms.

The remaining subsections reference material outlined in this article by T. Shchybovyk.

8.6.1 Why proofs are accessible and why hashing does not suffice

Proofs are public by design. In blockchain applications, to validate an off-chain event the prover submits the proof to a verifier—typically a smart contract. All on-chain activity is visible, so any proof submitted in a transaction is observable. An attacker can copy a proof from the mempool or from a past transaction.

Storing the proof (or its hash) to prevent reuse does *not* block the malleability attack: the forged proof $\pi' = (-[A]_1, -[B]_2, [C']_1)$ has a different hash than π , so a contract that rejects duplicate proof hashes will still accept π' . An airdrop that requires one proof per user can thus be exploited: a user submits a valid proof, an attacker observes it, negates $[A]_1$ and $[B]_2$, and submits the new proof to claim tokens again.

8.6.2 Geometric interpretation and EVM verification

On an elliptic curve, the negation $-R$ of a point R is its reflection over the x -axis (i.e., the y -coordinate is negated in the field). The pairing behaves like multiplication in the exponent: $e(-[A]_1, -[B]_2) = e([A]_1, [B]_2)$ because $(-1)(-1) = 1$. The attack therefore reduces to flipping the y -coordinates of the G_1 and G_2 components of A and B .

On Ethereum and EVM-compatible chains, the pairing precompile at address `0x8` aggregates several pairings and checks that their product equals the identity. The verification equation is rearranged so that all pairings are on one side and their product is compared to unity. The precompile expects $-A$ paired with B (or equivalent); the malleable proof $(-A, -B, C)$ still satisfies this rearranged check.

8.6.3 Mitigations

To prevent malleability-based double-claim or replay, applications should not rely on proof uniqueness alone. Common mitigations include: binding the proof to a nonce or message (e.g., a signature over the proof and context), using nullifiers to mark spent credentials, or tracking statements at the application layer rather than individual proofs. Using ZKPs without understanding these limitations can lead to exploitable designs.

Salt (Section 8.5) randomises proofs so that multiple proofs for the same witness are indistinguishable; it does *not* prevent replay. Any party who observes a valid proof can copy and resubmit it—the proof $\pi = (A, B, C')$ is public data. To prevent the same proof (or the same statement) from being accepted multiple times, applications must add their own mechanisms: e.g. include a nonce or session identifier in the public inputs, require a signature over the proof and context, or track which statements have already been proven at the application layer.

The prover can create unlimited proofs for the same witness; this is not a security flaw—it is necessary for zero-knowledge. With random r, s , each valid witness yields many possible proofs, so an attacker cannot link a proof to a specific witness. However, applications that need to track which facts have already been proven cannot rely on proof uniqueness to achieve that.

In summary: public inputs define the statement and commit the prover to a specific claim; without them, the verifier does not know what is being proven. The scalars γ and δ prevent the prover from lying about public inputs (Section 8.4), forcing correct separation of public and private contributions. The salts r, s prevent an attacker from linking a proof to a specific private witness when the witness has few valid values; they are sampled per proof by the prover, unlike α, β which are fixed at setup. Nonces, at the application level, prevent replay; salt does not. Finally, none of this is post-quantum: soundness rests on ECDLP and pairing-based assumptions, both broken by Shor’s algorithm on a sufficiently large quantum computer.

8.7 Groth16 Algorithm Summary

The full pipeline for constructing a ZK circuit to verify some logic: construct an arithmetic circuit encoding the logic of the programme; encode the circuit as a rank-1 constraint system (R1CS); interpolate its vectors to obtain a quadratic arithmetic program (QAP); perform the trusted setup, publishing the public components; and finally produce and verify a Groth16 proof that a claim satisfies the original circuit, in zero knowledge.

We now give the Groth16 algorithm end-to-end. The trusted setup and verification steps remain unchanged from the scheme with γ and δ (Section 8.4); only the prover’s calculation changes to incorporate the zero-knowledge salts r, s (Section 8.5).

The trusted setup samples $\alpha, \beta, \tau, \gamma, \delta$ from $\mathbb{F}_r$ uniformly, discards the scalars, and publishes:

- SRS for G_1 : $[\tau^{m-1}]_1, [\tau^{m-2}]_1, \dots, [\tau]_1, G_1$
- SRS for G_2 : $[\tau^{m-1}]_2, [\tau^{m-2}]_2, \dots, [\tau]_2, G_2$

- SRS for $H(\tau)t(\tau)/\delta$: $[\frac{\tau^{m-2}t(\tau)}{\delta}]_1, \dots, [\frac{t(\tau)}{\delta}]_1$
- Public Ψ_j ($j \leq \ell$): publish $[\Psi_j(\tau)/\gamma]_1$ where $\Psi_j = \alpha R_j + \beta L_j + O_j$
- Private Ψ_j ($j > \ell$): publish $[\Psi_j(\tau)/\delta]_1$
- Auxiliary: $[\alpha]_1, [\beta]_1, [\beta]_2, [\gamma]_2, [\delta]_1, [\delta]_2$

The prover, given witness $\mathbf{a} = (a_1, \dots, a_n)$ and random $r, s \in \mathbb{F}_r$, with $L(\tau) = \sum_{j=1}^n a_j L_j(\tau)$ and $R(\tau) = \sum_{j=1}^n a_j R_j(\tau)$:

$$\begin{aligned} [A]_1 &= [\alpha]_1 + L(\tau)G_1 + r[\delta]_1, \\ [B]_1 &= [\beta]_1 + R(\tau)G_1 + s[\delta]_1, \\ [B]_2 &= [\beta]_2 + R(\tau)G_2 + s[\delta]_2, \\ [C']_1 &= \sum_{j=\ell+1}^n a_j [\Psi_j(\tau)]_1 + [H(\tau)t(\tau)/\delta]_1 + s[A]_1 + r[B]_1 - rs[\delta]_1. \end{aligned}$$

Publish proof $\pi = ([A]_1, [B]_2, [C']_1)$ and public inputs $(a_1, \dots, a_\ell)$.

The verifier computes $[X]_1 = \sum_{j=1}^\ell a_j [\Psi_j(\tau)]_1$ from the public inputs and checks

$$e([A]_1, [B]_2) \stackrel{?}{=} e([\alpha]_1, [\beta]_2) \cdot e([X]_1, [\gamma]_2) \cdot e([C']_1, [\delta]_2). \quad (8.19)$$

A Proof of Fermat's Little Theorem

Fermat's Little Theorem is proven from base principles by introducing more rigorous notation around cyclic groups and subgroups, defining cosets and some properties, and finally, proving Lagrange's Theorem which can be conveniently used to prove Fermat's Little Theorem.

We wish to prove Fermat's Little Theorem in order to derive an identity which can transform a symbolic representation of an inverse element into a computable one.

Definition A.1. A group is cyclic if it can be generated by a single element. If a group H is generated by $x \in H$, we write $\langle x \rangle = H$.

Definition A.2. A subset $H \subseteq G$ is a *subgroup* of G , denoted $H \leq G$, if and only if $H \neq \emptyset$ and $\forall x, y \in H, xy^{-1} \in H$.

Definition A.3. If $H \leq G$, then for $g \in G$, the *left cosets of H in G* are denoted by the sets $gH = \{gh \mid h \in H\}$. Notably, distinct elements $g_1 \neq g_2$ may yield the same coset: $g_1H = g_2H$ if and only if $g_2^{-1}g_1 \in H$.

Definition A.4. A *partition* of G are nonempty sets A_i for indexing set I such that $\bigcup_{i \in I} A_i = G$ and $A_i \cap A_j = \emptyset$ for $i \neq j$, and $i, j \in I$.

Proposition A.1. For $H \leq G$, the left cosets of H in G form a partition of G .

Proof. $H \leq G \implies 1_G \in H$. Take g arbitrary, $g = g \cdot 1_G \in gH$, thus $G = \bigcup_{g \in G} gH$.

Additionally, for $g_1, g_2 \in G$, suppose $g_1H \cap g_2H \neq \emptyset$. Then, $\exists x \in g_1H \cap g_2H$, thus, for some appropriate $h_1, h_2 \in H$,

$$\begin{aligned} x &= g_1h_1 = g_2h_2 \\ \implies g_1 &= g_2(h_2h_1^{-1}) \in g_2H \end{aligned}$$

thus $g_1H \subseteq g_2H$. A similar argument can be used to show $g_2H \subseteq g_1H$. So, $\exists x \in g_1H \cap g_2H \implies g_1H = g_2H$. $\square$

Theorem A.2 (Lagrange's Theorem). If $H \leq G$, G finite, then $|H| \mid |G|$. That is, number of left cosets of H in G equals $\frac{|G|}{|H|}$.

Proof. Let $|H| = n$ and the number of left cosets of H in G be k . That is, for the set of cosets

$$K = \{g_{\alpha_1}H, g_{\alpha_2}H, \dots, g_{\alpha_k}H\}$$

where $g_{\alpha_i} \in G$ for $i = 1, \dots, k$ and the $g_{\alpha_i}H$ are the *distinct* left cosets, so $|K| = k$; different $g \in G$ may represent the same coset $g_{\alpha_i}H$. More formally, define a mapping

$$\begin{aligned} \pi : H &\rightarrow gH \\ h &\mapsto gh \end{aligned}$$

which, given $\forall gh \in gH \exists h \in H$ such that $\pi(h) = gh$, is a surjection from H to the left coset gH . Additionally, given $\pi(h_1) = \pi(h_2) \implies h_1 = h_2$, this mapping is also an injection of H onto gH , and thus π is a bijection. Namely,

$$|gH| = |H| = n$$

and by proposition A.1 we know G is partitioned by these k cosets of which each has n elements. Thus,

$$|G| = kn \implies |G| = k|H| \implies k = \frac{|G|}{|H|}$$

□

Theorem A.3 (Fermat's Little Theorem). This theorem is stated as Theorem 2.2. The proof is provided below.

Proof. If $p \mid a$, then $a^p \equiv 0 \equiv a \pmod{p}$ and the claim holds; so assume $p \nmid a$ and replace a by its residue modulo p . Consider $\mathbb{Z}_p^\times = \{1, 2, \dots, p-1\}$, the multiplicative group of integers modulo p , where p prime. Then $a \in \mathbb{Z}_p^\times$. Given $\mathbb{Z}_p^\times$ a group and finite, then

$$\langle a \rangle = \{a, a^2, \dots, a^d\}$$

where d is the least positive integer with $a^d = 1$, and the order of this group is $|\langle a \rangle| = d$. By Lagrange, we know that $|\langle a \rangle| \mid |\mathbb{Z}_p^\times|$. Thus $dk = p-1$ for some integer k , and $a^{dk} = a^{p-1} = 1^k = 1 = a^{p-1}$. Thus

$$\begin{aligned} a^{p-1} &\equiv 1 \pmod{p} \\ \iff a^p &\equiv a \pmod{p} \end{aligned} \tag{A.1}$$

□

Remark A.4 (Cyclicity of $\mathbb{Z}_p^\times$). An excellent monographical exposition of proofs of the cyclicity of the multiplicative group of integers modulo some prime number p can be found here, by K. Conrad. As a suggestion, examine 5. *FOURTH PROOF: INDUCTION AND HOMOMORPHISMS*, I, by D. Feldman and P. Monsky.

Having showed that $\mathbb{Z}_p^\times$ is a cyclic group, one can reason to $a^p \equiv a \pmod{p}$.

B Rational functions on curves: vocabulary and orders

The divisor formalism in Section 5 assumes that rational functions on a smooth projective curve are well behaved: they may have *zeros* and *poles* at isolated points, and each point P carries a signed integer $\text{ord}_P(f)$. This appendix collects the standard algebraic definitions in the same spirit as Silverman, *The Arithmetic of Elliptic Curves*, Ch. II, so that Definition 5.12 can be read with concrete meaning. For full treatments, see Silverman Ch. II §1 (affine and projective curves; function fields $K(C)$; local rings and uniformizers), §2 (maps between curves; used implicitly when changing coordinates), and especially §3 (divisors; principal divisors $\text{div}(f) = \sum_P \text{ord}_P(f)(P)$; degree and $\deg(\text{div}(f)) = 0$). Throughout, C denotes a smooth projective curve over an algebraically closed field (e.g. $C = E$ an elliptic curve over $\overline{\mathbb{F}}_p$).

B.1 Rational functions on a curve

To understand $\text{ord}_P(f)$, one only needs a small amount of language. A *rational function* on a curve C is a rule that assigns a field element to most points of C , but it may fail to be defined at some points. An *affine chart* is simply a patch of the curve on which one may use ordinary coordinates such as (x, y) . On such a chart, a *regular function* is an ordinary polynomial expression in those coordinates, so it is defined at every point of that chart. A rational function is obtained by allowing quotients of regular functions. Thus one usually writes

$$f = \frac{g}{h},$$

where g and h are regular functions on a chart and h is not the zero function (that is, h is not identically equal to 0 at every point of the chart). The collection of all rational functions on C forms a field, called the *function field* of C and denoted $K(C)$.

Remark B.1 (Explicit construction of $K(E)$). For an elliptic curve $E : y^2 = x^3 + ax + b$ over a field K , the function field can be built explicitly. The *coordinate ring* of the affine curve is the quotient

$$K[E] = \frac{K[x, y]}{(y^2 - x^3 - ax - b)},$$

i.e. one works with polynomials in x and y subject to the curve equation. Since E is smooth (hence irreducible), $K[E]$ is an integral domain, so one may form its fraction field:

$$K(E) = \text{Frac}(K[E]) = \text{Frac}\left(\frac{K[x, y]}{(y^2 - x^3 - ax - b)}\right).$$

Thus every element of $K(E)$ is a quotient of two polynomial expressions in x and y (modulo the curve relation), confirming that the informal description above—“quotients of regular functions”—matches the algebraic construction.

For elliptic curves, the affine and projective Weierstrass-coordinates for elliptic curves was discussed in Subsection 1.2. A projective is important here as a rational function may fail to be defined not only at affine points, but also at the point at infinity $\mathcal{O}$.

Example B.2. Let $K = \overline{\mathbb{F}}_p$ and let

$$E : y^2 = x^3 + ax + b$$

be an elliptic curve over K . Then x , y , and

$$f = \frac{y}{x-1}$$

are rational functions on E . At points where $x - 1 \neq 0$, the value of f is obtained by ordinary substitution. At a point with $x = 1$ and $y \neq 0$, the denominator is 0 while the numerator is not, so the quotient cannot be assigned a finite value there. This is the basic picture of a pole.

Remark B.3 (Terminology). When $K = \mathbb{C}$, rational functions on a smooth projective curve are often called *meromorphic* functions. In this appendix, the algebraic term *rational function* is enough, and no analytic background is needed.

This is the precise meaning of “function on a curve” used in Definition 5.12 and the divisor formalism of Section 5: an element of the function field $K(C)$, i.e. a quotient of polynomial expressions in the curve coordinates, subject to the curve equation. Such a function assigns a field element to all but finitely many points of C (the exceptions being its poles).

Definition B.1 (Regular points, zeros, and poles). Fix a point $P \in C$ and a rational function $f \in K(C)$.

1. f is *regular at P* if, after writing f in local coordinates near P , it can be assigned a finite value at P .
2. f *vanishes at P* if f is regular at P and $f(P) = 0$.
3. f is *nonvanishing at P* if f is regular at P and $f(P) \neq 0$.
4. f has a *pole at P* if it is not regular at P , that is, if near P it behaves like $1/u^m$ up to a nonzero regular factor for some positive integer m and some local coordinate u with $u(P) = 0$.

Thus, saying that f is regular and nonvanishing at P means exactly that f is finite and nonzero at P .

B.2 Zeros of multiplicity m , poles of order m , and the order $\text{ord}_P(f)$

Definition B.2 (Local parameter (uniformizer)). Let C be a smooth curve and $P \in C$. A *local parameter* (or *uniformizer*) at P is a rational function $u \in K(C)$ satisfying

1. $u(P) = 0$, and
2. every $f \in K(C)$ that is regular at P with $f(P) = 0$ can be written $f = uh$ with h regular at P ; that is, u generates the ideal $\mathfrak{m}_P$ of functions vanishing at P .

Intuitively, u is a one-dimensional coordinate centred at P : because the curve is smooth at P , a single function that vanishes there to first order is enough to parameterise the curve locally, and such a generator exists¹⁹.

Example B.4 (Concrete local parameters on $E : y^2 = x^3 + ax + b$). 1. *Generic affine point* $P = (x_0, y_0)$ with $y_0 \neq 0$: the function $u = x - x_0$ satisfies $u(P) = 0$ and has a simple zero at P , so it is a local parameter.

2. *2-torsion point* $P = (x_0, 0)$ (where the tangent is vertical): now $x - x_0$ vanishes to order 2 at P (the curve “turns around”), so $\text{ord}_P(x - x_0) = 2$ and $x - x_0$ is *not* a local parameter. Instead $u = y$ works, since y vanishes simply at such a point.

3. *Point at infinity* $\mathcal{O} = (0 : 1 : 0)$: neither x nor y is regular there (both have poles). A standard choice is $u = x/y$, which satisfies $\text{ord}_{\mathcal{O}}(x/y) = -2 - (-3) = 1$, making it a valid local parameter at $\mathcal{O}$.

To measure exactly how strongly f vanishes or fails to be finite at P , choose a local parameter u at P (Definition B.2). Then every nonzero rational function $f \in K(C)$ can be written uniquely in the form

$$f = u^m g \tag{B.1}$$

¹⁹Smoothness at P makes the local ring of functions regular at P a discrete valuation ring; see Silverman, *The Arithmetic of Elliptic Curves*, Ch. II, Proposition 1.1.

for some integer $m \in \mathbb{Z}$, where g is regular and nonvanishing at P . One defines

$$\text{ord}_P(f) = m.$$

This factorization is the precise meaning of the earlier informal language. The power u^m records all of the zero or pole behaviour at P , while g is the remaining part of the function and is finite and nonzero at P . In particular, $\text{ord}_P(u) = 1$ for the local parameter itself (take $m = 1$, $g = 1$), recovering the informal “simple zero” description.

1. If $m > 0$, then f has a *zero of multiplicity m* at P . Equivalently, f looks like u^m times something finite and nonzero.
2. If $m < 0$, then f has a *pole of order $-m$* at P . Equivalently, f looks like $1/u^{-m}$ times something finite and nonzero.
3. If $m = 0$, then f is regular and nonvanishing at P .

Thus, $\text{ord}_P(f)$ is a signed multiplicity: positive for zeros, negative for poles, and 0 when nothing special happens at P .

Remark B.5 (Independence of the choice of local parameter). If u and u' are two local parameters at P , then both vanish simply, so u'/u is regular and nonzero at P . Writing $u' = u \cdot (u'/u)$, one sees that replacing u by u' in the factorisation $f = u^m g$ merely absorbs the unit u'/u into g ; the exponent m is unchanged. Therefore $\text{ord}_P(f)$ is an intrinsic invariant of f at P , independent of which local parameter is used to compute it.

Example B.6. On the affine line, consider

$$f(x) = \frac{(x-a)^3}{x-b}.$$

Then f has a zero of multiplicity 3 at $x = a$, so

$$\text{ord}_a(f) = 3,$$

and it has a simple pole at $x = b$, so

$$\text{ord}_b(f) = -1.$$

At any point $c \neq a, b$, f is regular and nonvanishing (Definition B.1), so $\text{ord}_c(f) = 0$, i.e. the third case in (5.6).

Example B.7. Let E be an elliptic curve in Weierstrass form ($E : y^2 = x^3 + ax + b$). In homogeneous coordinates $(X : Y : Z)$ on $\mathbb{P}^2$, its projective closure is

$$Y^2Z = X^3 + aXZ^2 + bZ^3,$$

with affine chart $(x, y) = (X/Z, Y/Z)$ on the open set $Z \neq 0$. The identity (point at infinity) is $\mathcal{O} = (0 : 1 : 0)$: setting $Z = 0$ gives $X^3 = 0$ hence $X = 0$, and $Y \neq 0$ yields the unique point $(0 : 1 : 0)$, whereas $(0 : 0 : 1)$ is the affine origin in homogeneous form, not infinity—see Subsection 1.2 (and the caveat in Subsection 1.1 on $(0, 0)$ vs. $\mathcal{O}$ when $b \neq 0$). The coordinate functions x and y are rational functions on E . At $\mathcal{O}$, given $(X/Z, Y/Z)$, so undefined for $Z = 0$, one has

$$\text{ord}_{\mathcal{O}}(x) = -2, \quad \text{ord}_{\mathcal{O}}(y) = -3.$$

So x has a pole of order 2 at $\mathcal{O}$ and y has a pole of order 3 there. This is a standard elliptic-curve example showing that poles need not occur only on affine points; they may also occur at infinity in the projective model.

B.3 Principal divisors and degree

The principal divisor $\operatorname{div}(f) = \sum_{P \in C} \operatorname{ord}_P(f)(P)$ packages the local orders defined above into a single global object: the positive coefficients record zeros, the negative coefficients record poles. For the formal definition, the degree-zero property $\deg(\operatorname{div}(f)) = 0$, and a proof sketch, see Definition 5.13 and Theorem 5.8 in Section 5.